

Contents

Fundamentals of Programming Languages	1
Complete Topic List	1
Chapter Files	2
1.1 Lexical, Syntactic and Semantic Rules	2
1.2 Pragmatics	6
1.3 Compilation, Linkage and Interpretation	10
Cheat-Sheet	17
Review Checklist	17
2.1 Lexical Elements	17
2.2 Syntax: Arity and Fixity	20
2.3 Semantics	24
Cheat-Sheet	39
Review Checklist	40
3.1 Selection Statements (if, switch)	40
3.2 Iteration Statements (Loops)	47
3.3 Jump Statements	53
3.4 Pattern Matching in Control Flow	55
Cheat-Sheet	57
Review Checklist	57
4.1 Basic Types and Their Representation	57
4.2 Automatic and Explicit Type Conversions	64
 Chapter 4 Part 3: Arrays	 72
4.3 Compound Types: Arrays	72
4.4 Records and Structures	82
Cheat-Sheet	86
Review Checklist	86
4.5 Pointers	86
Cheat-Sheet	100
Review Checklist	100
5.1 Stack, Heap, Static (Lifetime)	100
5.2 Storing Variables in Memory and Lifetime	110
5.3 Parameter Passing, Execution Stack, Program Structure, Scope, and Visibility	112
Cheat-Sheet	123
Review Checklist	124
6.1 Raising and Propagation of Exceptions	124
6.2 Throwing and Handling Exceptions	128
6.3 Various Kinds of Exceptions	131
Cheat-Sheet	141
Review Checklist	141

Fundamentals of Programming Languages

Complete Topic List

1. Rules of a Programming Language

- Lexical, syntactic and semantic rules
- Pragmatics
- Compilation, linkage (or linking of code), and interpretation

2. Expressions and Their Evaluation

- Lexical elements: literals, identifiers, operators, parentheses
- Syntax: arity, fixity
- Semantics: Precedence, associativity (left-to-right, right-to-left), lazy and eager evaluation, side effect, order of evaluation, sequence point

3. Statements and Control Structures

4. Types

- Basic types and their representation
- Automatic and explicit type conversions
- Compound types: Arrays
- Records, structures
- Pointers

5. Memory Model and Subprograms

- Stack, heap, static (lifetime)
- Storing variables in memory and the lifetime of variables
- Parameter passing, execution stack, program structure, scope, and visibility

6. The Concept of Exceptions

- Raising and propagation of exceptions
- Throwing and handling exceptions
- Various kinds of exceptions

Chapter Files

- chapters/chap1_rules.md
- chapters/chap2_expressions.md
- chapters/chap3_control_structures.md
- chapters/chap4_types.md
- chapters/chap5_memory_model.md
- chapters/chap6_exceptions.md # Chapter 1: Rules of a Programming Language

1.1 Lexical, Syntactic and Semantic Rules

Definition / Theoretical Background

Programming languages are defined by three layers of rules that govern how code is written, interpreted, and executed. Understanding these three rule types is fundamental to mastering any language.

Lexical Rules (The “Spelling” of Language) Lexical rules define how characters are grouped into meaningful tokens—the smallest units of a program. Think of it as the spelling and word formation rules of a natural language.

Source Code Character Stream

```
->
[Lexer/Scanner]
->
```

Token Stream: KEYWORD "if", IDENTIFIER "score", NUMBER "42", OPERATOR "+"

Token Types:

Token Type	Description	Examples
Keywords/Reserved Words	Words with special meaning, cannot be reused	if, while, return, class
Identifiers	User-defined names for variables, functions	count, calculateTotal, maxValue
Literals	Fixed values that don't change	42, 3.14, "Hello", true
Operators	Operations to perform	+, -, *, /, ==, &&
Delimiters/Punctuation	Structural separators	(,), {, }, ;, ,
Comments	Non-executable documentation	// line comment, /* block comment */

Syntactic Rules (The “Grammar” of Language) Syntactic rules define how tokens combine to form valid statements and programs. This is the grammar-correct sentence structure.

```
<if_statement> ::= "if" "(" <expression> ")" <statement>
<expression>  ::= <identifier> <relational_op> <expression>
<statement>   ::= <if_statement> | <assignment> | <while_loop> | ...
```

Key Concepts:

- **Context-Free Grammar (CFG):** Most programming languages use CFG for syntax
- **Derivation:** Starting from the start symbol, repeatedly apply production rules until you get the token sequence
- **Parse Tree:** Visual representation of how a program can be derived from the grammar

```

    <if_statement>
      |
    -----
    |           |
  "if"      ( <expression> ) <statement>
              |
            <ident> < <expression>
              |           |
            "x"         <ident>
                       |
                       "10"
```

Semantic Rules (The “Meaning” of Language) Semantic rules define what programs mean—what they actually do when executed.

Aspect	Example
Type Rules	<code>int x = "hello";</code> – type mismatch error
Scope Rules	Using a variable before declaration
Context Sensitivity	<code>*</code> means multiplication for numbers, dereference for pointers
Evaluation Order	<code>a + b * c</code> – which addition happens first?

Edge Cases & Traps

Trap 1: Reserved Words vs Identifiers

```
// C/C++ -- sizeof is a keyword, cannot be used as variable name
int sizeof = 5; // ERROR: cannot declare identifier "sizeof"

// But in some languages like SQL, you can sometimes escape reserved words
SELECT "select" FROM "table"; // Works in some SQL dialects
```

Trap 2: Unicode and Identifiers

```
# Python allows Unicode identifiers (PEP 3131)
caf? = 42 # Valid in Python 3
delta = 100 # Also valid (Greek letter as identifier)
# But not all languages support this!
```

Trap 3: Context-Sensitive Syntax

```
// JavaScript: / can mean division OR regex depending on context
let x = y / 2; // Division: y divided by 2
let pattern = /[a-z]+/; // Regex literal -- context determines meaning!
```

Trap 4: Invisible Characters in Source

```
// The following looks identical but one has a non-breaking space
int x = 5; // Valid
int y = 5; // Invalid -- invisible character after 5 (U+200B)
```

[WARNING] **Professor Trap:** “This code looks fine but won’t compile” – always check for hidden Unicode characters!

Examples

```
int main() {
    float score = 95.5;
    if (score >= 60) {
        return 1;
    }
}
```

```
    return 0;
}
```

Example 1: Breaking Down Simple Code Lexically Tokenization:

KEYWORD:int	KEYWORD:main	SYMBOL:(	SYMBOL:)
KEYWORD:float	IDENTIFIER:score	OPERATOR:=	FLOAT:95.5
KEYWORD:if	SYMBOL:(	IDENTIFIER:score	OPERATOR:>=
NUMBER:60	SYMBOL:)	SYMBOL:{	KEYWORD:return
NUMBER:1	SYMBOL;;	SYMBOL:}	KEYWORD:return
NUMBER:0	SYMBOL;;	SYMBOL:}	

```
# Python interprets this as: (3 + 4) * 5 due to precedence
result = 3 + 4 * 5 # result = 23, not 35
```

```
# But what about this?
x = a if True else b if False else c
# Parsed as: x = a if True else (b if False else c)
```

Example 2: Syntactic Ambiguity

Practice Questions

MCQ 1: What type of rule determines whether `int x = "hello";` is valid? - A) Lexical rule - B) Syntactic rule - C) Semantic rule - D) Pragmatic rule

MCQ 2: Which of the following is NOT a valid token type? - A) Identifier - B) Reserved word - C) Semantic marker - D) Operator

Short Answer: Explain why `x = y` is syntactically valid but semantically invalid in the context of `int x; float y = 3.14; x = y;`.

Board Coding: Write the token stream for: `printf("Result: %d", a + b * c);`

Solutions

MCQ 1: C) Semantic rule – type checking happens during semantic analysis.

MCQ 2: C) “Semantic marker” is not a standard token type.

Short Answer: The assignment `x = y` follows correct syntactic structure (variable = expression). However, assigning a `float` to an `int` variable violates semantic rules (type compatibility). The compiler may warn or error depending on language settings.

Board Coding Solution:

```
IDENTIFIER:printf SYMBOL:( STRING:"Result: %d" SYMBOL:,
IDENTIFIER:a OPERATOR:+ IDENTIFIER:b OPERATOR:* IDENTIFIER:c
SYMBOL:)
```

1.2 Pragmatics

Definition / Theoretical Background

Pragmatics refers to the practical aspects of writing code—conventions, style, and formatting that make code readable and maintainable. While a program may be syntactically and semantically correct, poor pragmatics makes it unusable by others.

“Code is read far more often than it is written.” – Guido van Rossum

Pragmatics includes:

1. **Naming Conventions:** How we name variables, functions, classes
2. **Code Formatting:** Indentation, whitespace, line length
3. **Documentation:** Comments, docstrings, README files
4. **Coding Standards:** Language-specific idioms and patterns

Naming Conventions

Common Naming Styles

Style	Example	Used In
camelCase	maxValue, calculateTotal	Java, JavaScript
PascalCase	MaxValue, CalculateTotal	C#, TypeScript classes
snake_case	max_value, calculate_total	Python, Ruby, C
SCREAMING_SNAKE_CASE	MAX_VALUE, BUFFER_SIZE	Constants (universal)
kebab-case	max-value, calculate-total	Lisp, CSS, URLs

```
// JavaScript/TypeScript conventions
const MAX_BUFFER_SIZE = 1024;      // Constants: SCREAMING_SNAKE
let userName = "Alice";           // Variables: camelCase
class UserAccount { }             // Classes: PascalCase
function calculateTotalPrice() { } // Functions: camelCase

// Python conventions (PEP 8)
MAX_BUFFER_SIZE = 1024             # Constants (module-level)
user_name = "Alice"                # Variables: snake_case
class UserAccount: pass            # Classes: PascalCase
def calculate_total_price(): pass   # Functions: snake_case
```

Industry Examples

Code Formatting & Style

```
// C/C++/Java style: 4 spaces or tab
void function() {
    if (condition) {
        doSomething();
    }
}
```

```

    }
}

// Python style: 4 spaces (required!)
def function():
    if condition:
        do_something()

```

Indentation [WARNING] **Professor Trap:** Mixing tabs and spaces causes subtle bugs!

```

# This looks fine but contains a tab mixed with spaces
def broken():
    ****if True:      # 4 spaces
    [TAB]print("hi") # tab -- Python sees indentation error!

# Always use consistent indentation (spaces recommended: 4 per level)

```

```

# Bad: Extremely long lines
result = some_function(argument1, argument2, argument3, argument4, argument5, argument6, argument7)

# Good: Broken into readable chunks
result = some_function(
    argument1,
    argument2,
    argument3,
    argument4,
    argument5,
    argument6,
    argument7
)

```

Line Length & Readability

Code Formatters (Automated Pragmatics)

Modern development uses automated tools to enforce formatting:

```

# Prettier (JavaScript/TypeScript) -- "An opinionated code formatter"
npx prettier --write src/*.js

# Black (Python) -- "The uncompromising code formatter"
black main.py

# clang-format (C/C++) -- "Format your code automatically"
clang-format -i source.c

// Prettier configuration (.prettierrc)
{

```

```

"semi": true,
"singleQuote": true,
"tabWidth": 2,
"trailingComma": "es5",
"printWidth": 80
}

```

Documentation Pragmatics

```

# Bad: Obvious comment stating what code does
# Increment x by 1
x = x + 1

# Good: Comment explains WHY, not WHAT
x = x + 1 # Compensate for 1-indexed array in legacy API

# Excellent: Docstring for functions
def calculate_average(numbers):
    """
    Calculate the arithmetic mean of a list of numbers.

    Args:
        numbers: List of int or float values

    Returns:
        float: The arithmetic mean, or 0 if list is empty

    Raises:
        TypeError: If numbers contains non-numeric types
    """
    if not numbers:
        return 0
    return sum(numbers) / len(numbers)

```

Edge Cases & Traps

Trap 1: Meaningless Variable Names

```

// Professor will mark this down!
// What does x, y, z represent?
int x = calculate(x, y, z);

// Better: Descriptive names
int total_price = calculate(unit_price, quantity, tax_rate);

```

Trap 2: Inconsistent Naming in Same Codebase

```
// Confusing: mixing conventions in one file
let userName = "Alice";      // camelCase
let user_age = 25;           // snake_case -- INCONSISTENT!
let UserAddress = {};        // PascalCase for variable -- unusual!
```

Trap 3: Comments That Lie

```
# Professor trap: Comments don't match code!
x = 10 # Initialize to 0
# Later in code...
x = 20 # Programmer forgot to update comment
```

Trap 4: Cultural Pragmatics

```
// Variable names that might confuse international teams
int basket_price = 0;      // "basket" or "shopping cart"?
int colour = RED;          // British vs American spelling inconsistency
int???? = 0;               // Unicode in variable names -- not portable!
```

Examples

```
// BEFORE (poor pragmatics)
function p(x,y){let r=x*y;if(r>100){return r*.15}else{return r*.1}}

// AFTER (good pragmatics)
function calculateDiscount(price, quantity) {
    const subtotal = price * quantity;
    const discountRate = subtotal > 100 ? 0.15 : 0.10;
    return subtotal * discountRate;
}
```

Example 1: Pragmatic Code Review

```
// Avoid: ASCII-only assumptions that fail internationally
char filename[] = "caf?_r?sum?.txt"; // May cause issues
// Better: Use UTF-8 consistently
const char* filename = "cafe_resume.txt"; // Safe, portable

// Or properly handle Unicode
std::string filename = u8"caf?_r?sum?.txt"; // C++11 UTF-8 string
```

Example 2: Internationalization Pragmatics

Practice Questions

MCQ 1: Which naming convention is typically used for constants in Python? - A) camelCase - B) snake_case - C) SCREAMING_SNAKE_CASE - D) PascalCase

MCQ 2: What is the main purpose of code formatters like Prettier? - A) Check for syntax errors - B) Enforce consistent formatting automatically - C) Optimize runtime performance - D) Convert between programming languages

Short Answer: Why might a professor deduct points for code that compiles and runs correctly but uses poor naming conventions?

Oral Explanation: Explain the difference between syntactic rules and pragmatic conventions. Why might code that follows syntactic rules still be considered “bad code”?

Solutions

MCQ 1: C) SCREAMING_SNAKE_CASE (e.g., MAX_SIZE, BUFFER_LENGTH)

MCQ 2: B) Enforce consistent formatting automatically

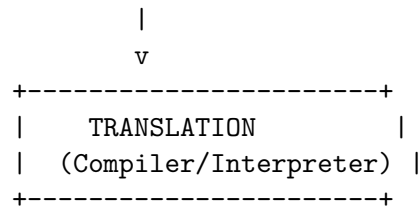
Short Answer: Because pragmatics affects code maintainability, readability, and collaboration. In professional settings, code is read by many people. Poor naming makes code harder to understand, debug, and maintain—skills that are evaluated in real-world programming.

1.3 Compilation, Linkage and Interpretation

Definition / Theoretical Background

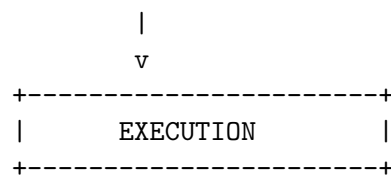
Understanding how source code becomes an executable program is crucial for every programmer.

Source Code (.c, .py, .java)



```

      |
      v
Machine Code / Intermediate Form
```



Compilation vs Interpretation

Aspect	Compiler	Interpreter
Process	Translates entire program before execution	Translates and executes line-by-line
Output	Standalone executable (.exe, .out)	No separate output, runs directly

Aspect	Compiler	Interpreter
Speed	Fast execution (optimized native code)	Slower (translation at runtime)
Debugging	Harder (harder to trace runtime errors)	Easier (can stop and inspect)
Examples	C, C++, Go, Rust	Python, Ruby, JavaScript, PHP

```
// hello.c
#include <stdio.h>

int main() {
    printf("Hello, World!\n");
    return 0;
}
```

Compiler Workflow (C/C++ Example)

```
hello.c (source)
    |
    v [Preprocessor]
hello.i (preprocessed)
    |
    v [Compiler]
hello.s (assembly)
    |
    v [Assembler]
hello.o (object file - machine code)
    |
    v [Linker]
hello (executable)
```

Types of Compilation

1. Ahead-of-Time (AOT) Compilation The traditional compilation model where code is fully compiled before execution.

```
# C/C++ example
gcc -o program source.c    # Compile to native executable
g++ -O2 -o program source.cpp # With optimization
```

Characteristics: - Fastest possible execution - Platform-specific binaries - No runtime overhead

2. Just-in-Time (JIT) Compilation Compiles code at runtime, often used to improve performance of interpreted languages.

```
// Java HotSpot JIT compiler
// JVM compiles frequently-used bytecode to native code at runtime
// Methods are compiled after they are called many times ("hot")
```

```
// JavaScript V8 engine (Chrome/Node.js)
// Ignition interpreter + Turbofan JIT compiler
function hotFunction() { /* called 10000 times */ }
```

Source Code

|

v

Bytecode (intermediate)

|

+--- [Interpreter: Ignition] --- executes bytecode initially

|

+--- [JIT Compiler: Turbofan] --- compiles hot paths to native code

|

v

Native Code (cached)

3. Hybrid Approaches Modern languages often combine multiple strategies:

Language	Strategy
Java	Source -> Bytecode (.class) -> JVM (interpreter) + JIT
Python	CPython (pure interpreter) vs PyPy (JIT)
C#/.NET	Source -> IL bytecode -> CLR (interpreter + JIT)
JavaScript	V8: Parser -> Bytecode (Ignition) -> JIT (Turbofan)
WebAssembly	Binary format that runs in browser (AOT or JIT via browser engine)

Linkage (Linking)

Linkage is the process of combining multiple object files and libraries into a single executable.

Types of Linkage

Static Linking Libraries are copied into the final executable at compile time.

```
# Static linking example (Linux)
gcc -static -o program source.c
# Creates large executable with all library code included

# Result: Larger file, but self-contained
```

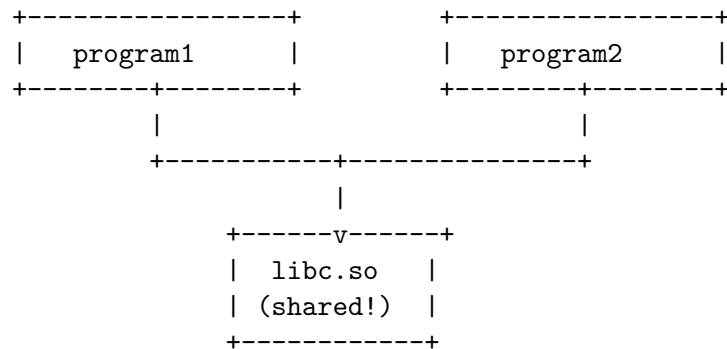
Advantages: - No dependency on system libraries - Slightly faster (no runtime library lookup)

Disadvantages: - Larger file size - Cannot update libraries without recompiling - Memory duplication across programs

Dynamic Linking Libraries are loaded at runtime, shared across programs.

```
# Dynamic linking (default on most systems)
gcc -o program source.c      # Links to system .so files

# On Linux: .so (Shared Object)
# On Windows: .dll (Dynamic Link Library)
# On macOS: .dylib (Dynamic Library)
```



Advantages: - Smaller executables - Shared memory across programs - Library updates without recompilation

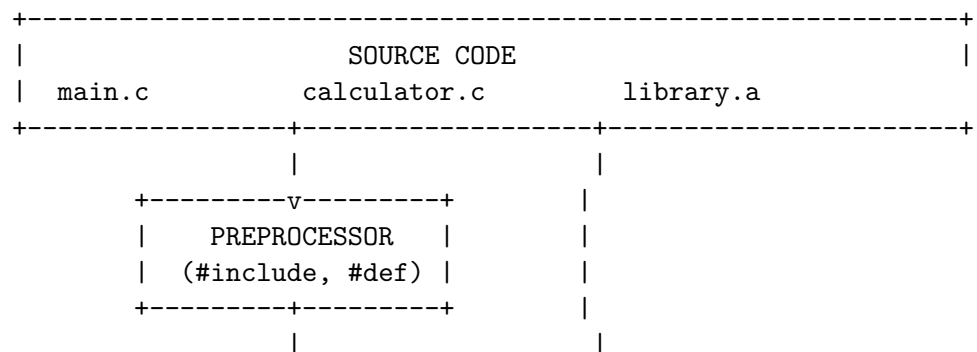
Disadvantages: - “DLL Hell” – version conflicts - Dependency on correct library versions - Slightly slower startup

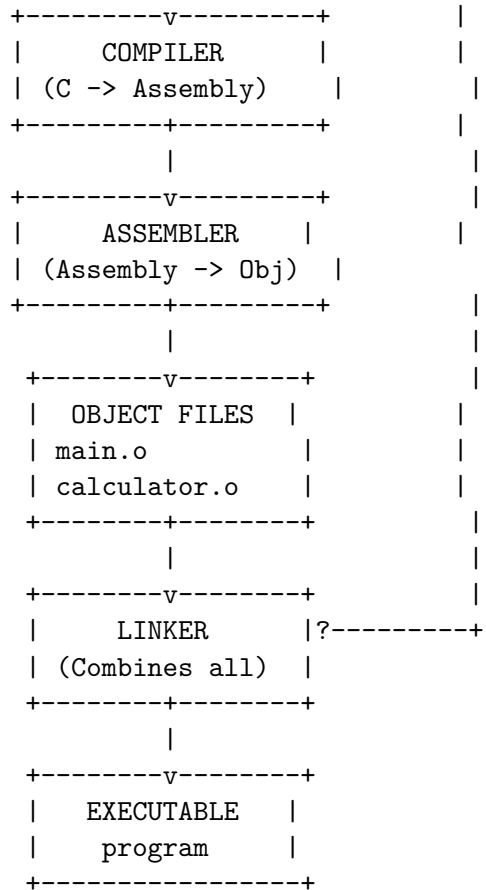
Dynamic Loading Program loads libraries on-demand at runtime:

```
// dlopen example (POSIX systems)
#include <dlfcn.h>

void *handle = dlopen("./libcalc.so", RTLD_LAZY);
if (handle) {
    int (*calc)(int, int) = dlsym(handle, "calculate");
    result = calc(10, 20);
    dlclose(handle);
}
```

The Complete Build Process





Assembly and Bytecode Comparison

Aspect	Assembly	Bytecode
Abstraction	Very low-level, architecture-specific	Higher-level, virtual machine
Execution	Direct CPU execution	Virtual machine interprets
Portability	Platform-specific (x86, ARM)	Cross-platform (JVM, .NET CLR)
Verification	None (direct memory access)	Security checks by VM
Example	C/C++ native code	Java .class, Python .pyc

Edge Cases & Traps

Trap 1: Linker Symbol Conflicts

```

// file1.c
int counter = 0;

// file2.c
double counter = 3.14; // ERROR: multiple definition of 'counter'

```

```
// Fix: Use static or proper namespaces
static int counter = 0; // file1.c -- internal linkage
// OR use extern in header
```

Trap 2: Header File Multiple Inclusion

```
// shapes.h
#ifndef SHAPES_H
#define SHAPES_H
struct Point { int x, y; };
#endif

// Without include guards, linking fails with "redefinition"
```

Trap 3: Static vs Dynamic Library Version Mismatch

```
/usr/lib/libssl.so.1.1    ? Your program needs this
/usr/lib/libssl.so.1.0    ? System has this
```

[WARNING] **Professor Trap:** “The program compiles but crashes at runtime” – check library dependencies with `ldd` (Linux) or `dumpbin` (Windows)!

Trap 4: Bytecode Interpretation vs JIT Behavior

```
# CPython (interpreter): True and True returns True (boolean)
>>> True and True
True

# But in C with JIT-like optimization, might short-circuit differently
# Be aware: Python's short-circuit evaluation is guaranteed
def foo(): print("foo"); return True
def bar(): print("bar"); return False
result = foo() and bar()
# Prints: foo (bar() never called due to short-circuit)
```

Examples

```
# Check executable dependencies (Linux)
ldd ./my_program
# Output:
#  linux-vdso.so.1 (0x00007ffd...)
#  libc.so.6 => /lib64/libc.so.6 (0x00007f2a...)
#  libm.so.6 => /lib64/libm.so.6 (0x00007f2a...)
#  libpthread.so.0 => /lib64/libpthread.so.0 (0x00007f2a...)

# Check symbols in object file
nm ./my_program | head -20
# Output:
# 0000000000201010 T main
```

```
# 0000000000201020 T calculate
#                               U printf
```

Example 1: Examining a Compiled Program

```
// V8 (Node.js) shows JIT behavior
function add(a, b) {
    return a + b;
}

// Call many times -- V8 will JIT compile this "hot" function
for (let i = 0; i < 1000000; i++) {
    add(i, i + 1); // After ~1000 calls, V8 may compile to native
}

// You can inspect with: node --trace-opt --trace-deopt script.js
```

Example 2: JIT Compilation Demonstration

Practice Questions

MCQ 1: Which type of compilation happens at runtime, often improving performance of hot code paths? - A) AOT (Ahead-of-Time) - B) JIT (Just-in-Time) - C) Static compilation - D) Preprocessing

MCQ 2: What is the main advantage of dynamic linking over static linking? - A) Faster execution - B) Smaller executable size - C) No runtime dependencies - D) Direct CPU execution

MCQ 3: In the compilation pipeline, what does the assembler produce? - A) Source code - B) Assembly code - C) Object file (machine code) - D) Executable

Short Answer: Explain why Java uses both a compiler and an interpreter. Why not compile directly to machine code?

Board Coding: Draw the complete compilation pipeline for a C program that uses `#include <stdio.h>` and produces an executable.

Oral Explanation: Compare JIT compilation vs interpretation. Under what circumstances might JIT compilation provide better performance?

Solutions

MCQ 1: B) JIT (Just-in-Time) – compiles at runtime based on execution frequency.

MCQ 2: B) Smaller executable size – libraries are not copied into the executable.

MCQ 3: C) Object file (machine code) – the assembler converts assembly to binary object files.

Short Answer: Java compiles to bytecode (platform-independent), then the JVM interprets or JIT-compiles it. This provides portability—you can run the same .class file on any platform with a JVM. Direct native compilation would require separate executables for each platform.

Cheat-Sheet

Concept	Key Points
Lexical Rules	Tokenization, token types (keywords, identifiers, literals, operators, delimiters)
Syntactic Rules	Grammar (BNF), derivation, parse trees, grammar correctness
Semantic Rules	Type checking, scope, meaning of code, context-sensitive analysis
Pragmatics	Naming conventions (camelCase, snake_case, SCREAMING_SNAKE), formatting, documentation
AOT Compilation	Full compilation before execution; fast runtime, platform-specific
JIT Compilation	Runtime compilation of hot paths; balances startup speed and optimization
Static Linking	Libraries embedded in executable; larger size, no dependencies
Dynamic Linking	Libraries loaded at runtime (.so/.dll/.dylib); smaller executables, shared memory
Dynamic Loading	On-demand library loading via dlopen/GetProcAddress

Review Checklist

- ☐ **Definitions:** Can you define lexical, syntactic, and semantic rules?
- ☐ **Token Types:** Can you identify all token types in sample code?
- ☐ **Pragmatics:** Do you know all naming conventions and when to use each?
- ☐ **Compilation Types:** Can you explain AOT vs JIT and give examples?
- ☐ **Linkage Types:** Can you contrast static, dynamic linking, and dynamic loading?
- ☐ **Build Pipeline:** Can you trace source -> executable through all stages?
- ☐ **Trap Questions:** Are you aware of common professor traps (reserved words, invisible characters, symbol conflicts)?
- ☐ **Practical:** Can you use tools like `ldd`, `nm`, and `objdump` to inspect binaries? # Chapter 2: Expressions and Their Evaluation

2.1 Lexical Elements

Definition / Theoretical Background

An **expression** is a combination of values, variables, operators, and function calls that a programming language evaluates to produce a result. The building blocks of expressions are **lexical elements**.

Types of Lexical Elements 1. **Literals** Fixed values that represent themselves:

```
42          # Integer literal
3.14159     # Floating-point literal
```

```
"Hello"    # String literal
'c'        # Character literal
True       # Boolean literal (Python)
nullptr    # Null pointer (C++)
```

2. Identifiers Names that refer to variables, functions, classes:

```
int maxValue = 100;           // maxValue is identifier
float calculateTotal();       // calculateTotal is identifier
class UserAccount {};
```

3. Operators Symbols that specify operations:

Category	Operators
Arithmetic	+, -, *, /, %, **, //
Comparison	==, !=, <, >, <=, >=
Logical	&&, , !, and, or, not
Bitwise	&, , ^, ~, <<, >>
Assignment	=, +=, -=, *=, /=
Ternary	? :

4. Parentheses Grouping symbols that affect evaluation order:

```
result = (a + b) * c;    // Parentheses force a+b to evaluate first
```

Edge Cases & Traps

Trap 1: Integer vs Float Division

```
// C/C++ integer division
int a = 5, b = 2;
printf("%d", a / b);    // Output: 2 (not 2.5!)

// To get float division
printf("%f", (float)a / b);    // Output: 2.500000
```

[WARNING] **Professor Trap:** “Why does 5/2 equal 2?” – Integer literals in integer division truncate!

Trap 2: String vs Character Literals

```
// In C/C++
"a"    // String literal: pointer to memory containing 'a' and '\0' (2 bytes)
'a'    // Character literal: integer value 97 (1 byte)

// This will compile but cause confusion
printf("%c", "a"[0]);    // Prints 'a' -- works but is confusing code!
```

[WARNING] **Professor Trap:** "a" == 'a' is FALSE – different types!

Trap 3: Floating-Point Literals

```
# Python
0.1 + 0.2 == 0.3    # FALSE! Due to floating-point representation
# 0.1 + 0.2 = 0.30000000000000004
```

[WARNING] **Professor Trap:** Never compare floats with `==` – use tolerance!

```
import math
math.isclose(0.1 + 0.2, 0.3)    # True
```

Trap 4: Underscores in Numeric Literals

```
// C++14 and later
int million = 1'000'000;    // Valid: easier to read
int bytes   = 0b1010'0010;  // Binary with separator
```

[WARNING] **Professor Trap:** 1,000,000 (with commas) is NOT valid – only single quotes!

Trap 5: Scientific Notation Interpretation

```
# Different interpretations across languages
1e10    # Python: 1 * 10^10 = 10000000000
1e-5    # Python: 1 * 10^-5 = 0.00001

// JavaScript: same as Python
// C: same, but 'e' must be lowercase
```

Examples

```
// Expression: result = (x + 5) * y - z / 2 + func(a, b);
//
// Token Stream:
// IDENTIFIER:result OPERATOR:= SYMBOL:( IDENTIFIER:x OPERATOR:+ NUMBER:5 SYMBOL:)
// OPERATOR:* IDENTIFIER:y OPERATOR:- IDENTIFIER:z OPERATOR:/ NUMBER:2
// OPERATOR:+ IDENTIFIER:func SYMBOL:( IDENTIFIER:a SYMBOL:, IDENTIFIER:b SYMBOL:) SYMBOL;;
```

Example 1: Breaking Down Complex Expressions

```
// Common escape sequences
"Hello\nWorld"    // Newline
"Path\tC:\\Program" // Tab and backslash
"Quote: \"Hello\"" // Double quote
"NULL\0"          // Null terminator
```

Example 2: String Literal Escape Sequences

Practice Questions

MCQ 1: What is the result of `5 / 2` in C? - A) 2.5 - B) 2 - C) 2.0 - D) Error

MCQ 2: Which of the following is NOT a lexical element? - A) Identifier - B) Keyword - C) Parse tree - D) Operator

Short Answer: Explain the difference between a string literal "a" and a character literal 'a' in C.

Board Coding: Tokenize the following expression: `float result = (x - 5.0) / 2.0 + calculate(y);`

Solutions

MCQ 1: B) 2 – integer division truncates

MCQ 2: C) Parse tree – this is a syntactic structure, not a lexical element

Short Answer: "a" is a string literal representing an array of 2 characters ('a' and '\0'), stored at a memory address. 'a' is a character literal representing the integer value 97. They have different types and sizes.

2.2 Syntax: Arity and Fixity

Definition / Theoretical Background

Arity The **arity** of an operator or function is the number of operands it requires.

Operator Type	Arity	Description	Examples
Nullary	0	No operands	true, false, NULL, literals
Unary	1	Single operand	-x, !flag, &ptr, *ptr, i++, ++i
Binary	2	Two operands	a + b, x && y, a < b
Ternary	3	Three operands	condition ? a : b
N-ary (Variadic)	Variable	Any number of operands	max(a,b,c,d), printf(fmt, ...), sum(1,2,3,4,5)

Variadic Function Examples:

```
# Python: max() can take ANY number of arguments
```

```
max(1, 5, 3, 9, 2)      # Returns 9
```

```
max(10, 20)             # Returns 20
```

```
max(42)                  # Returns 42
```

```
# C: variadic printf
```

```
printf("Hello %s, age=%d", name, age); # Format string + variable args
```

```
# JavaScript: Math.max with spread
```

```
Math.max(...[1, 5, 3, 9, 2]); // Returns 9
```

```
# C++: variadic templates
template<typename... Args>
int sum(Args... args) { return (... + args); }
sum(1, 2, 3, 4, 5); // Returns 15
```

Fixity Fixity determines the position of operators relative to their operands.

Prefix: Operator before operands

```
-x           // Unary minus (prefix)
!flag        // Logical NOT (prefix)
-5           // Negation (prefix)
```

Infix: Operator between operands (most common)

```
a + b        // Binary infix
x > y         // Comparison infix
a ? b : c     // Ternary infix
```

Postfix: Operator after operands

```
i++          // Post-increment
array[j++]   // Postfix in array access
factorial()  // Factorial (some languages)
```

Operator Fixity Examples

```
# Prefix operators in Python
not True      # Logical NOT (prefix)
-5            # Unary minus (prefix)
abs(-10)      # Function call (prefix notation)

# Postfix operators
i++           # C/C++ post-increment
factorial(5)! # Some functional languages

# Infix (most common)
a + b
x > 5
```

Edge Cases & Traps

Trap 1: Unary vs Binary Minus Ambiguity

```
// What does this mean?
x = -5;           // Unary minus applied to literal 5
y = 3 - -5;       // Binary minus, then unary minus on 5: y = 8
z = 3--5;         // ERROR? Actually valid: z = 8 (binary minus, unary minus)
```

[WARNING] **Professor Trap:** $3--5$ is $3 - (-5) = 8$, not an error!

Trap 2: Nested Unary Operators

```
int x = 5;
int y = ---x;    // Valid? Three unary minuses = unary minus of (unary minus of (unary minus of
                  // Equivalent to  $-(-(-x)) = -(-5) = 5$ 
                  // Compiles but style is terrible!
```

[WARNING] **Professor Trap:** Three --- in a row compiles because -- is post-decrement only when there's an operand before it!

Trap 3: Function Call Arity Mismatch

```
// C -- no runtime arity check
int add(int a, int b) { return a + b; }
add(1, 2, 3);    // Compiles! (extra arguments ignored -- undefined behavior in some contexts)
// Warning: too many arguments
```

[WARNING] **Professor Trap:** C allows calling with wrong number of arguments – always check prototypes!

Trap 4: Ternary Operator Nesting

```
// What is this?
int x = a ? b ? c : d : e;

// Parsed as: a ? (b ? c : d) : e
// NOT: (a ? b : c) ? d : e
```

[WARNING] **Professor Trap:** Ternary operator is right-associative!

Examples

```
// Given: a + b * c - d / e % f
// Arity analysis:
// + : binary (a + ...)
// * : binary (... * ...)
// - : binary (... - ...)
// / : binary (... / ...)
// % : binary (... % ...)

// All infix binary operators here
```

Example 1: Identifying Arity

```
# Prefix notation: function name before arguments
result = max(10, 20, 30)    # Prefix: max is function name

# Some languages use prefix for everything
# Lisp: (+ 3 4) -- prefix, operator first
# x = (+ (* 2 3) 4) -- equivalent to x = (2*3) + 4
```

Example 2: Prefix vs Postfix in Function Calls

White-Board Walkthrough: Arity Parsing

Expression: $a + b * c - d / e$

Step 1: Identify all operators and their positions

```
a  +  b  *  c  -  d  /  e
      ^      ^      ^      ^
      binary binary binary binary
```

Step 2: Build operator tree (lowest precedence at top)

```
      -
    +-----+-----+
      +               /
+-----+-----+ +---+
a       *      d   e
    +-----+
    b       c
```

Step 3: Evaluate bottom-up (post-order traversal)

```
b * c = result1
d / e = result2
a + result1 = result3
result3 - result2 = final
```

Practice Questions

MCQ 1: How many operands does the ternary operator `?:` require? - A) 1 - B) 2 - C) 3 - D) 4

MCQ 2: Which is a postfix operator? - A) `-x` - B) `!flag` - C) `i++` - D) `-5`

Short Answer: Explain why `3--5` is valid C but `3-5` (missing minus) would not give the same result.

Board Coding: Draw the parse tree for $a + b * c - d$ and show evaluation order.

Solutions

MCQ 1: C) 3 – condition ? value_if_true : value_if_false

MCQ 2: C) `i++` – postfix means operator after operand

Short Answer: `3--5` parses as $3 - (-5) = 8$ because the first `-` is binary subtraction and the second `-5` is unary minus applied to 5. They are different operators. `3-5` parses as $3 - 5 = -2$. Different meaning.

2.3 Semantics

Definition / Theoretical Background

Semantics defines the meaning of expressions – how they are evaluated, what values they produce, and what side effects they may have.

Key Semantic Concepts:

1. **Precedence:** Which operators are evaluated first
2. **Associativity:** How operators of same precedence are grouped
3. **Evaluation Order:** Left-to-right vs right-to-left
4. **Lazy vs Eager Evaluation:** When arguments are evaluated
5. **Side Effects:** Changes to state beyond the return value
6. **Sequence Points:** Points where all previous evaluations are complete

Precedence

Operator precedence determines which operators are evaluated first when an expression has multiple operators.

Complete Precedence Table (C/Java/JavaScript)

Level	Operators	Associativity
1	() [] . ->	Left-to-right
2	! ~ ++ -- - (unary) * (dereference) & (address) sizeof	Right-to-left
3	* / %	Left-to-right
4	+ -	Left-to-right
5	<< >>	Left-to-right
6	< <= > >=	Left-to-right
7	== !=	Left-to-right
8	& (bitwise AND)	Left-to-right
9	^ (bitwise XOR)	Left-to-right
10	(bitwise OR)	Left-to-right
11	&&	Left-to-right
12	\ \	Left-to-right
13	?: (ternary)	Right-to-left
14	= += -= *= /= %= &= ^= \ = <<= >>=	Right-to-left
15	, (comma)	Left-to-right

Associativity

When operators have the same precedence, associativity determines grouping.

```
// LEFT-TO-RIGHT associativity (most operators)
a + b + c    // Parsed as: (a + b) + c
a - b - c    // Parsed as: (a - b) - c  <- IMPORTANT: NOT a - (b - c)!
a * b * c    // Parsed as: (a * b) * c
a / b / c    // Parsed as: (a / b) / c
```

CONCRETE EXAMPLE - Left-to-Right:

Expression: $10 - 4 - 2$

Step 1: Parse according to associativity
 $(10 - 4) - 2$ <- LEFT-TO-RIGHT for subtraction

Step 2: Evaluate inside parens
 $6 - 2$

Step 3: Final result
4

WRONG ANSWER (if you did right-to-left):
 $10 - (4 - 2) = 10 - 2 = 8$ <- This is INCORRECT!

Answer: 4

Professor Trap with Subtraction:

```
// What is 10 - 4 - 2?  
int x = 10 - 4 - 2; // x = 4 (left-to-right)  
// If parsed as 10 - (4 - 2) = 10 - 2 = 8, WRONG!
```

Another Example - Division:

```
// What is 100 / 10 / 2?  
int y = 100 / 10 / 2;  
// Parsed as: (100 / 10) / 2 = 10 / 2 = 5  
// NOT: 100 / (10 / 2) = 100 / 5 = 20  
  
printf("%d", y); // Prints 5  
  
// RIGHT-TO-LEFT associativity (assignment, ternary)  
a = b = c // Parsed as: a = (b = c)  
a ? b ? c : d : e // Parsed as: a ? (b ? c : d) : e
```

CONCRETE EXAMPLE - Right-to-Left Assignment:

Expression: $a = b = 5$

Step 1: Parse according to associativity
 $a = (b = 5)$ <- RIGHT-TO-LEFT for assignment

Step 2: Evaluate rightmost first
 $b = 5$ -> b is now 5, returns 5

Step 3: $a = 5$
a is now 5

Final state: $a = 5, b = 5$

```
// Concrete C example
int a, b, c;
a = b = c = 100;

// Step-by-step:
// c = 100 -> c = 100
// b = 100 -> b = 100
// a = 100 -> a = 100

// Final: a=100, b=100, c=100
```

Why Right-to-Left for Assignment?

```
// This makes sense: right side determines left side
int x, y, z;
x = y = z = calculate(); // All get same value!

// If it were left-to-right:
// temp = x = y (assign y to x, use result to assign to z?)
// Confusing!
```

Ternary Right-to-Left Example:

```
// What is: a ? b ? c : d : e ?
// Right-to-left: a ? (b ? c : d) : e

// If a = 1, b = 1, c = 10, d = 20, e = 30
// 1 ? (1 ? 10 : 20) : 30
// = 1 ? (1 ? 10 : 20) : 30
// = 1 ? 10 : 30
// = 10
```

[WARNING] **Professor Trap:** Ternary is right-associative! $a?b:c?d:e = a?(b:c)?d:e = a?(b?(c:d):e)?$

Evaluation Order

Critical Question: In $a + b * c$, is a evaluated before or after $b * c$?

The answer depends on the language: - **C/C++:** Order of evaluation of operands is **unspecified** (except for certain operators) - **Java/Python:** Left-to-right order is guaranteed

```
// C: DANGER - evaluation order not specified!
int i = 1;
int arr[10] = {0};
int x = arr[i] + (i = 2);
// Is this arr[1] + 2 or arr[2] + 2? UNDEFINED!
```

[WARNING] **Professor Trap:** This code has undefined behavior in C!

Lazy vs Eager Evaluation

KEY CONCEPT: When are the arguments to a function/expression evaluated?

Eager Evaluation (Strict/Call-by-Value) ALL arguments are evaluated BEFORE the function is called.

```
# Python uses EAGER evaluation
def expensive(x):
    print(f"Computing with {x}")
    return x * 2

expensive(5 + 3) # First: 5+3=8 is computed, THEN expensive(8) is called
# Output: "Computing with 8"
```

```
// C uses EAGER evaluation for function arguments
int square(int x) {
    printf("Computing %d\n", x);
    return x * x;
}

int result = square(2 + 3); // First: 2+3=5 computed, THEN square(5) called
// Output: "Computing 5"
// result = 25
```

Lazy Evaluation (Non-Strict/Call-by-Need) Arguments are evaluated ONLY when they are actually needed.

```
-- Haskell uses LAZY evaluation
expensive :: Int -> Int
expensive x = x * x

-- If we write: expensive (5 + 3)
-- 5+3 is NOT computed until it's actually used inside expensive!
-- This is "call by need" - delayed evaluation
```

PRACTICAL EXAMPLE - Short-Circuit is a form of Lazy Evaluation:

```
# Python's and/or operators use short-circuit (lazy)
def expensive():
    print("expensive called!")
    return False

# With and:
result = False and expensive() # expensive() is NEVER called!
# Short-circuit: False and anything = False, so right side skipped

# What about or?
result = True or expensive() # expensive() is NEVER called!
# Short-circuit: True or anything = True, so right side skipped
```

```
result = False or expensive()  # expensive() IS called
# Short-circuit: False or ... needs to evaluate right side
```

COMMON EXAM PATTERN - Default Value:

```
# Lazy evaluation for default values
def get_user_name(name=None):
    if name is None:
        name = "Guest"  # Only computed when needed
    return name

# Eager equivalent would wastefully compute default:
# def get_user_name(name=None):
#     name = "Guest"  # Always computed (wasteful if name provided)
```

LAZY LISTS (Important for Functional Languages):

```
# Python generators are LAZY
def infinite_sequence():
    i = 0
    while True:
        yield i
        i += 1

# Even though infinite, doesn't consume memory until used!
# gen = infinite_sequence()
# next(gen)  # Returns 0, computes only when needed
# next(gen)  # Returns 1
```

JAVA SCRIPT LAZY EVALUATION EXAMPLE:

```
// JavaScript: // can provide default values (lazy behavior)
let name = userName || "Guest";
// If userName is falsy (null, undefined, ""), "Guest" is used
// But if userName is provided, "Guest" is never evaluated

// What about this?
let x = expensive() || fallback();
// expensive() is called FIRST, then based on result, fallback may or may not be called
```

C COMPARISON - Eager vs Lazy:

```
// C ALWAYS evaluates eagerly (except short-circuit with &&, ||, ?:)
// No lazy evaluation built-in

// But we can simulate lazy behavior:
int result = a && b;  // Short-circuit: b only evaluated if a is true
// This IS a form of lazy evaluation!

// The && operator is lazy in C!
```

KEY DIFFERENCES TABLE:

Language	Evaluation Strategy	Notes
C/C++	Eager	Arguments evaluated before call (except &&,
Python	Eager	Arguments evaluated before call
Java	Eager	Arguments evaluated before call
Haskell	Lazy	Arguments evaluated only when needed
JavaScript	Eager	But short-circuit operators are lazy

```
# PRACTICAL EXAM QUESTION: What is printed?
def test(x, y):
    print(f"x={x}, y={y}")
    return x + y

# In Python (eager - all evaluated):
result = test(2 + 3, 4 + 5) # First: 2+3=5, 4+5=9, THEN test(5, 9) called
# Prints: x=5, y=9
# result = 14

# But in lazy language (like Haskell):
-- The arguments might not be evaluated until actually used inside the function
```

Side Effects

A **side effect** is any change to program state beyond the computed value: - Modifying a variable - Printing output - Reading input - Modifying a global variable - Throwing an exception

```
// Side effects in action
int counter = 0;

int increment() {
    return counter++; // Returns current value, THEN increments (side effect!)
}

// Side effect confusion
int x = counter++; // x gets old value, counter is incremented
```

Sequence Points (CRITICAL FOR EXAMS)

A **sequence point** is a point in execution where ALL side effects from previous evaluations must be complete.

SEQUENCE POINTS IN C (Memorize these!):

Sequence Point	Example	Description
;	a++; b++;	End of statement
&&	a() && b()	Short-circuit AND

Sequence Point	Example	Description
\ \	a() \ \ b()	Short-circuit OR
?:	a ? b : c	Ternary operator
, (operator)	(a++, b++)	Comma operator only
Function call	func(a++, b++)	After all arguments evaluated

KEY RULE: Between sequence points, a variable can only be modified ONCE!

EXAMPLES OF UNDEFINED BEHAVIOR (No sequence point between operations):

```
// UNDEFINED #1: i = i++
int i = 5;
i = i++; // UNDEFINED! i modified twice: once by =, once by ++
// Between start and ; there's no sequence point separating the modifications

// UNDEFINED #2: printf with multiple ++
int j = 1;
printf("%d %d", j++, j++); // UNDEFINED!
// Which j++ is evaluated first? Order unspecified!
// Possible outputs: "1 2" or "2 1" - compiler-dependent!

// UNDEFINED #3: i++ + ++i
int k = 5;
int result = i++ + ++i; // UNDEFINED! i modified twice without sequence point
// See Trap 6 for full explanation

// UNDEFINED #4: array index with assignment
int arr[5] = {0, 1, 2, 3, 4};
int idx = 0;
arr[idx] = idx++; // UNDEFINED! idx modified and accessed without sequence point
// arr[0] = arr[0]? or arr[1] = arr[0]? Nobody knows!
```

EXAMPLES OF DEFINED BEHAVIOR (With sequence points):

```
// DEFINED #1: Semicolon separates
int i = 5;
i++; // Sequence point here (semicolon)
i++; // This is fine - different statements

// DEFINED #2: Short-circuit
int x = 1, y = 2;
if (x++ && y++) {
    // x++ is evaluated first
    // If x++ is non-zero, y++ is evaluated
    // Sequence point after && determines order
}
// After if, both modifications are complete
```

```
// DEFINED #3: Comma operator
int a = (a = 5, a + 1); // Defined! Comma is a sequence point
// First a=5 executes, then a+1, then assignment to a
// a = 6

// DEFINED #4: Function arguments (sort of)
func(i++, i++); // UNDEFINED in C! Arguments may be evaluated in any order
// But AFTER func is called, i's value is well-defined (if 2 increments happened)
```

PRACTICAL EXAMPLE - Why This Matters:

```
// WRONG - Undefined behavior
int i = 1;
printf("%d", i++ * i++); // UNDEFINED!

// RIGHT - Defined behavior
int i = 1;
int x = i++; // x = 1, i = 2
int y = i++; // y = 2, i = 3
printf("%d", x * y); // Prints 2
```

COMMON PROFESSOR QUESTION:

```
// What is the result of:
int i = 1;
i = i++;
printf("%d", i);

// Answer: UNDEFINED! i is modified twice without sequence point.
// Any compiler may output: 1, 2, or something else!
```

WHEN ARE SEQUENCE POINTS IMPORTANT?

1. Multiple modifications to same variable in one expression
2. Function arguments in C (order not specified)
3. Short-circuit evaluation ensures left-to-right order
4. Ternary operator has sequence point after condition

```
// All of these are UNDEFINED in C:
i = i++ + ++i;
arr[i] = i++;
printf("%d %d", i++, i++);
a = i = i++;

// All of these are DEFINED:
i++; i++; // Sequential statements with ;
result = (a++, b++); // Comma operator
if (a++ && b++) // && is sequence point
a = b ? c++ : d++; // ?: is sequence point
```

Edge Cases & Traps

Trap 1: Precedence Confusion with Bitwise and Comparison

```
int x = 5;
int result = x & 0x0F == 0;  // WRONG interpretation!

// & has lower precedence than ==? NO!
// == has higher precedence than &
// Actually parsed as: x & (0x0F == 0)
// Which is: x & 0 (because 0x0F != 0)
// Result: 0

// Correct:
result = (x & 0x0F) == 0;  // Parens around bitwise operation
```

[WARNING] **Professor Trap:** Bitwise operators (&, |, ^) have LOWER precedence than comparisons (==, <, >)!

Trap 2: Assignment vs Comparison

```
// C/C++: assignment is an expression
if (x = 5) { // Sets x to 5, then checks if 5 is truthy
    printf("x is 5 or non-zero\n");
}

// Common bug: when you meant to compare
if (x = y) // Bug: assigns, doesn't compare!
```

[WARNING] **Professor Trap:** if (x = 5) is VALID code, not a syntax error!

Trap 3: Short-Circuit Evaluation (IMPORTANT FOR EXAMS)

```
// THE isValidUser && isAdmin PATTERN (very common!)
bool isValidUser = false;
bool isAdmin = true;

// If user is not valid, admin check is SKIPPED
if (isValidUser && isAdmin) {
    showDashboard(); // NOT called because isValidUser is false!
}

// STEP-BY-STEP with && (short-circuit):
// Step 1: Evaluate isValidUser -> false
// Step 2: false && anything = false (SHORT-CIRCUIT! Right side NOT evaluated)
// Step 3: showDashboard() is NEVER called

// WHY? If isValidUser is false, doesn't matter what isAdmin is
// The whole condition is false anyway
```

```
// THE || PATTERN (short-circuit on TRUE)
bool isLoggedIn = true;
bool hasPermission = checkPermission();

// If already logged in, permission check is SKIPPED
if (isLoggedIn || hasPermission) {
    accessGranted(); // Called immediately!
}

// Step-by-step with ||:
// Step 1: Evaluate isLoggedIn -> true
// Step 2: true || anything = true (SHORT-CIRCUIT! Right side NOT evaluated)
// Step 3: accessGranted() IS called
```

COMMON EXAM PATTERN - NULL CHECK:

```
// WRONG way (crashes if ptr is NULL):
if (ptr->value > 0 && ptr != NULL) // Order matters! Check ptr first!

// RIGHT way (safe):
if (ptr != NULL && ptr->value > 0) { // SAFE: if ptr is NULL, right side skipped
    // Only enters if ptr is NOT NULL
}

// WRONG way with bitwise (& instead of &&):
if (ptr != NULL & ptr->value > 0) { // DANGER! Both sides always evaluated!
    // CRASHES if ptr is NULL!
}
```

The Critical Difference:

Operator	Type	Short-Circuits?	Both sides evaluated?
&&	Logical AND	YES	Only left if left is false
&	Bitwise AND	NO	ALWAYS both

```
// EXAM QUESTION: What does this print?
int x = 0;
int arr[5] = {10, 20, 30, 40, 50};

if (x < 5 && arr[x] == 10) {
    printf("First element is 10");
} else {
    printf("Not found");
}

// Answer: "Not found"
// x = 0, x < 5 is true
```

```

// But wait... arr[0] = 10, so arr[x] == 10 should be true...
// Actually prints "First element is 10"!

// Let's try with x = 5:
x = 5;
if (x < 5 && arr[x] == 10) { // x < 5 is FALSE
    printf("First element is 10");
} else {
    printf("Not found"); // PRINTED! Short-circuit!
}
// Because x < 5 is false, arr[x] is never accessed!

// ANOTHER CRITICAL EXAMPLE - Division by Zero Protection:
int denominator = 0;

if (denominator != 0 && 10 / denominator > 1) {
    printf("Result is greater than 1");
} else {
    printf("Result is not greater than 1 or division by zero prevented");
}
// Prints: "Result is not greater than 1 or division by zero prevented"
// 10/denominator is NEVER evaluated!

// But with single & (bitwise):
if (denominator != 0 & 10 / denominator > 1) { // CRASH! Division by zero!
    printf("...");
}

```

[WARNING] **Professor Trap:** && vs & – first short-circuits (SAFE), second evaluates BOTH sides ALWAYS (DANGEROUS)!

Trap 4: Comma Operator Confusion

```

// Comma operator: evaluates left, discards, evaluates right, returns right
int x = (5, 10); // x = 10

// BUT: function arguments separated by commas are NOT the comma operator
printf("%d, %d", x, y); // Two separate arguments, not comma operator

// Sequence point after first argument evaluation is guaranteed
// But evaluation order of arguments is still unspecified!

```

Trap 5: Floating Point Precedence

```

# Python: what is 0.1 + 0.2?
result = 0.1 + 0.2
print(result) # 0.30000000000000004

# NOT a precedence issue, but precision issue

```

[WARNING] **Professor Trap:** Floating point is never exact – never use == for float comparison!

Trap 6: The Famous i++ + ++i Trap (CRITICAL FOR EXAM)

```
int i = 5;
int result = i++ + ++i;

// STEP-BY-STEP ANALYSIS:
// The expression `i++ + ++i` has TWO side effects on i with NO sequence point between them!

// i++ (post-increment):
//   1. Returns the CURRENT value of i (5)
//   2. THEN increments i to 6

// ++i (pre-increment):
//   1. First increments i to 7
//   2. THEN returns the new value (7)

// But the ORDER of evaluation of left and right operands is UNSPECIFIED in C!

// POSSIBLE EVALUATION ORDERS:

// Case 1: Left operand evaluated first
//   Step 1: Evaluate left side (i++)
//     i++ returns 5, i becomes 6
//   Step 2: Evaluate right side (++i)
//     ++i returns 7, i becomes 7
//   Step 3: result = 5 + 7 = 12

// Case 2: Right operand evaluated first
//   Step 1: Evaluate right side (++i)
//     ++i returns 7, i becomes 7
//   Step 2: Evaluate left side (i++)
//     i++ returns 7, i becomes 8
//   Step 3: result = 7 + 7 = 14

// Case 3: GCC might inline the values
//   Some compiler: 5 + 7 = 12

// REALITY: COMPILER CAN DO ANYTHING! THIS IS UNDEFINED BEHAVIOR!
```

[WARNING] **Professor Trap:** THIS IS UNDEFINED BEHAVIOR IN C/C++! Never write code like this on exams!

More dangerous ++ traps:

```
// TRAP 6a: Array access with side effect
int i = 1;
int arr[10] = {0, 1, 2, 3, 4, 5, 6, 7, 8, 9};
int x = arr[i] + (i = 2);
```

```

// What is arr[i]? Is it arr[1]=1 or arr[2]=2?
// UNDEFINED! No sequence point between array index and assignment!

// TRAP 6b: Double modification
int j = 5;
j = j++; // UNDEFINED! j is modified twice without sequence point
// Never do this!

// TRAP 6c: Prefix ++ in expression
int k = 5;
int z = ++k + ++k + ++k;
// UNDEFINED! k is modified 3 times with no sequence points!
// Even though mathematically (6+7+8) = (7+6+8) = (8+6+7) = 21
// Compiler might give: 6+7+8 = 21, or 7+6+8 = 21, or 8+7+6 = 21, or something else!

```

TRAP 6d: The Difference Between Prefix and Postfix

```

int i = 5, j = 5;

// Prefix ++i: increment FIRST, return NEW value
int a = ++i; // i becomes 6, a = 6

// Postfix i++: return OLD value, THEN increment
int b = j++; // b = 5, j becomes 6

// This matters in expressions:
int x = 5;
int result = x++ + x++;
// UNDEFINED but IF it worked:
// If evaluated left-to-right: (5 + 6) = 11, x ends at 7
// If evaluated right-to-left: (6 + 5) = 11, x ends at 7
// Mathematically same here, but NOT guaranteed!

// Prefix in expressions - also UNDEFINED:
int y = 5;
result = ++y + ++y + ++y;
// UNDEFINED! y modified multiple times without sequence point!
// Different compilers will give DIFFERENT answers!

```

[WARNING] **Professor Trap:** When ++ or -- appear multiple times in the same expression, the behavior is UNDEFINED unless there are parentheses ensuring order!

Examples

```

// Evaluate: result = 3 + 4 * 5 - 6 / 3 + (2 + 3) * (4 - 1)

// Step 1: Parentheses first
// (2 + 3) = 5

```

```

// (4 - 1) = 3

// Step 2: Multiplication/division left-to-right
// 4 * 5 = 20
// 6 / 3 = 2

// Step 3: Now we have
// result = 3 + 20 - 2 + 5 * 3

// Step 4: Multiplication
// 5 * 3 = 15

// Step 5: Addition/subtraction left-to-right
// 3 + 20 = 23
// 23 - 2 = 21
// 21 + 15 = 36

// result = 36

```

Example 1: Complex Precedence Evaluation

```

// C: Short-circuit evaluation
int divide(int a, int b) {
    printf("divide called\n");
    return a / b;
}

int x = 1;
int y = 0;

if (x != 0 && divide(10, y) > 5) { // SAFE!
    // divide() is NOT called because x != 0 is false
    // && short-circuits
}

// Same with ||
if (x == 0 || divide(10, y) == 10) { // divide is NOT called!
    // Left side of || is false, so right side evaluated?
    // NO! || short-circuits when left side is true
    // Here left is FALSE, so right side IS evaluated, divide is called
}

```

Example 2: Short-Circuit Logic

```

int a, b, c;
a = b = c = 10;

```

```
// Right-to-left: parsed as a = (b = (c = 10))  
// 1. c = 10 -> c is 10  
// 2. b = 10 -> b is 10  
// 3. a = 10 -> a is 10  
  
// Result: a = 10, b = 10, c = 10
```

Example 3: Right-to-Left Associativity in Assignment

White-Board Walkthrough: Evaluation Order

Question: What is the result of: `printf("%d", (1, 2, 3));`

Step 1: Comma operator -- left to right

(1, 2, 3) -> evaluate 1 (discard), evaluate 2 (discard), evaluate 3 (return)

Step 2: The entire expression returns 3

Step 3: `printf` prints 3

Answer: 3

Question: What is the result of: `int x = 5; x = x++ + ++x;`

Step 1: This is UNDEFINED BEHAVIOR in C!

Reason: x is modified twice without an intervening sequence point:

- `x++` modifies x (returns 5, then increments to 6)
- `++x` modifies x (increments to 7, returns 7)
- No sequence point between these operations

Possible outcomes:

- Some compilers: $5 + 7 = 12$
- Some compilers: $6 + 6 = 12$
- Some compilers: $5 + 6 = 11$
- Something else entirely!

Answer: UNDEFINED -- NEVER write code like this!

Practice Questions

MCQ 1: In C, what is `3 + 4 * 5` equal to? - A) 35 - B) 23 - C) 17 - D) 15

MCQ 2: Which operators short-circuit in C? - A) `&` and `|` - B) `&&` and `||` - C) `&` and `||` - D) `&&` and `|`

MCQ 3: What does `int x = (1, 2, 3);` assign to x? - A) 1 - B) 2 - C) 3 - D) Undefined

Short Answer: Explain why `x = x++` is undefined behavior in C.

Board Coding: Evaluate step by step: `int result = 2 + 3 * 4 - 8 / 2 + (5 - 3);`

Oral Explanation: Explain the difference between precedence, associativity, and evaluation order. Why is it important to understand each concept?

Solutions

MCQ 1: B) `23` – `*` has higher precedence than `+`, so `3 + (4 * 5) = 3 + 20 = 23`

MCQ 2: B) `&&` and `||` – logical operators short-circuit

MCQ 3: C) `3` – comma operator evaluates left-to-right and returns the last value

Short Answer: In `x = x++`, the expression `x++` modifies `x` (post-increment) and returns the old value. But before this is assigned to `x`, the increment also happens. There's no sequence point between the read-modify-write of `x`. The language doesn't specify whether the increment happens before or after the assignment, leading to undefined behavior.

Board Coding Solution:

Step 1: Parentheses $\rightarrow (5 - 3) = 2$

Step 2: Multiplication/division $\rightarrow 3 * 4 = 12, 8 / 2 = 4$

Step 3: Expression becomes $\rightarrow 2 + 12 - 4 + 2$

Step 4: Left-to-right $\rightarrow 2 + 12 = 14$

Step 5: $14 - 4 = 10$

Step 6: $10 + 2 = 12$

Result: 12

Cheat-Sheet

Concept	Key Points
Literals	Fixed values: integers, floats, strings, characters, booleans
Identifiers	User-defined names; must follow language naming rules
Arity	Number of operands: unary (<code>-x</code>), binary (<code>a+b</code>), ternary (<code>a?b:c</code>), variadic
Fixity	Prefix (<code>++i</code>), Infix (<code>a+b</code>), Postfix (<code>i++</code>)
Precedence	Higher precedence = evaluated first; use parentheses to override
Associativity	Same precedence: L-to-R for most operators; R-to-L for assignment/ternary
Evaluation Order	C: unspecified for operands; Python/Java: left-to-right
Short-Circuit	<code>&&</code> and <code> </code> don't evaluate right side if unnecessary
Side Effects	Modifications to state beyond return value
Sequence Points	Points where all side effects complete: <code>;</code> , <code>&&</code> , <code> </code> , <code>?:</code> , function call

Concept	Key Points
Lazy Evaluation	Arguments evaluated only when needed (Haskell, some JS)
Eager Evaluation	All arguments evaluated before function call (Python, C)

Operator Precedence Quick Reference (C/Java)

Highest: () [] . ->	Left-to-right
! ~ ++ -- + - * &	Right-to-left (unary)
* / %	Left-to-right
+ -	Left-to-right
<< >>	Left-to-right
< <= > >=	Left-to-right
== !=	Left-to-right
& (bitwise)	Left-to-right
^ (bitwise XOR)	Left-to-right
(bitwise OR)	Left-to-right
&&	Left-to-right
	Left-to-right
?:	Right-to-left
= += -= *= /= %=	Right-to-left
Lowest: ,	Left-to-right

Review Checklist

- ☐ **Literals:** Can you identify integer, float, string, character, boolean literals?
- ☐ **Arity:** Can you distinguish unary, binary, ternary, variadic operators?
- ☐ **Fixity:** Can you identify prefix, infix, and postfix operators?
- ☐ **Precedence:** Can you correctly parenthesize `a + b * c - d / e`?
- ☐ **Associativity:** Can you explain why `a = b = c` assigns right-to-left?
- ☐ **Evaluation Order:** Do you know which languages guarantee left-to-right evaluation?
- ☐ **Short-Circuit:** Can you explain why `ptr != NULL && ptr->value` is safe?
- ☐ **Side Effects:** Can you identify expressions with side effects?
- ☐ **Sequence Points:** Do you know all sequence points in C?
- ☐ **Trap Questions:** Are you aware of common traps (precedence with bitwise, undefined behavior with ++, float comparison)? # Chapter 3: Statements and Control Structures

3.1 Selection Statements (if, switch)

Definition / Theoretical Background

Selection statements allow the program to choose between different paths of execution based on conditions.

The if Statement

```

// Simple if
if (condition) {
    // code executed if condition is true
}

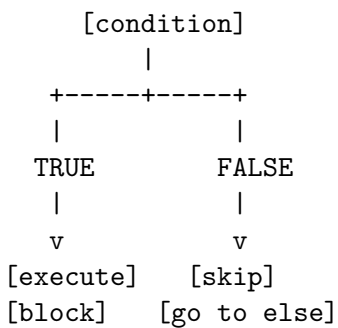
// if-else
if (condition) {
    // true branch
} else {
    // false branch
}

// if-else if-else chain
if (condition1) {
    // code 1
} else if (condition2) {
    // code 2
} else if (condition3) {
    // code 3
} else {
    // default code
}

```

Basic Syntax

How if Works (Flowchart)



Condition Evaluation In C, a condition is **true** if the expression evaluates to **non-zero**, and **false** if it evaluates to **zero**.

```

int x = 10;
if (x) { printf("True"); } // Prints "True" (x is non-zero)
if (x > 5) { printf("True"); } // Prints "True" (x > 5 is true)

if (!0) { printf("True"); } // !0 = 1 (true)
if (!NULL) { printf("True"); } // NULL is 0, !0 = 1 (true)

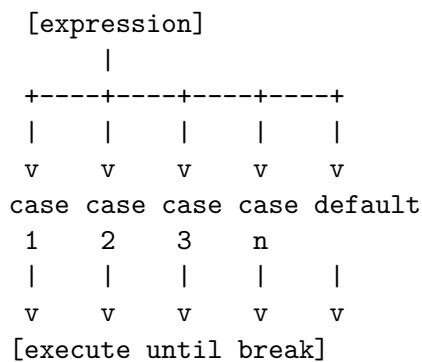
```

The switch Statement

```
switch (expression) {
    case constant1:
        // code for case 1
        break;
    case constant2:
        // code for case 2
        break;
    // ... more cases
    default:
        // code if no case matches
        break;
}
```

Syntax

Switch Flow Diagram



Switch Rules

1. **Expression must evaluate to integer type** (int, char, enum, not float/string)
2. **Case labels must be constants** or constant expressions
3. **No two case labels can be the same**
4. **break is optional** - without it, execution “falls through”

```
char grade = 'B';
switch (grade) {
    case 'A':
        printf("Excellent");
        break;
    case 'B':
        printf("Good");
        break;          // Without break, would fall through to C!
    case 'C':
        printf("Average");
        break;
    default:
```

```
        printf("Invalid grade");
    }
```

Edge Cases & Traps

Trap 1: Assignment Inside if (Common Bug!)

```
// WRONG - assigns x to 5, then checks if 5 is true
if (x = 5) {
    // This always executes because 5 is non-zero (true)!
}

// CORRECT - comparison
if (x == 5) {
    // Only executes if x equals 5
}

// C style check that some use (intentional assignment + immediate check):
if ((x = getValue()) > 0) {
    // x is assigned, then checked
}
```

[WARNING] **Professor Trap:** `if (x = 5)` is VALID C, not a syntax error! It's an assignment disguised as comparison!

Trap 2: Using = vs ==

```
// This is a common bug
int flag = 0;
if (flag = 1) { // Bug! Sets flag to 1, always true!
    printf("Flag is set");
}

// Safer style - literal on left:
if (1 == flag) { // Bug! If you use =, compiler catches it!
    // This would be a compile error if you wrote 1 = flag
}
```

[WARNING] **Professor Trap:** Some put the constant on the LEFT to catch accidental assignment!

Trap 3: Missing break (Fall-through)

```
int digit = 2;
switch (digit) {
    case 0:
        printf("Zero");
    case 1:
        printf("One");
    case 2:
        printf("Two"); // Falls through intentionally (bug or feature?)
    case 3:
```

```

        printf("Three");
        break;
    default:
        printf("Other");
}
// Output: TwoThree (cases 2 and 3 executed because no break after case 2!)

```

[WARNING] **Professor Trap:** Missing break causes fall-through! Sometimes intentional (like for a-f handling), but usually a bug.

Trap 4: Switch with Float

```

float f = 1.5;
switch (f) { // ERROR! Switch expression must be integer type!
    case 1.5: // Can't use float case labels either
        break;
}

```

[WARNING] **Professor Trap:** Switch only works with integer types (int, char, enum), NOT float or double!

Trap 5: Multiple Same Labels

```

int x = 2;
switch (x) {
    case 1:
        printf("One");
        break;
    case 2:
        printf("Two");
        break;
    case 2: // ERROR! Duplicate case label
        printf("Two again");
        break;
}

```

Trap 6: Unintended Execution of Default

```

int x = 10;
switch (x) {
    case 10:
        printf("Ten ");
    default:
        printf("Default"); // Also prints because no break!
}
// Output: Ten Default

```

Trap 7: Initialization Inside Case

```

switch (x) {
    case 1:

```

```

        int y = 5; // WARNING: declaration in block - some compilers complain
        break;
    case 2:
        y = 10;    // ERROR! y not in scope here
        break;
}

// Fix: use braces to create scope
switch (x) {
    case 1: {
        int y = 5;
        break;
    }
    case 2: {
        int y = 10;
        break;
    }
}

```

Examples

```

// Grade calculation
int score = 85;
char grade;

if (score >= 90) {
    grade = 'A';
} else if (score >= 80) {
    grade = 'B';
} else if (score >= 70) {
    grade = 'C';
} else if (score >= 60) {
    grade = 'D';
} else {
    grade = 'F';
}

printf("Grade: %c\n", grade); // B

```

Example 1: if-else Chain

```

int choice;
printf("1. New Game\n2. Load Game\n3. Settings\n4. Quit\n");
scanf("%d", &choice);

switch (choice) {
    case 1:

```

```

        printf("Starting new game...\n");
        break;
    case 2:
        printf("Loading saved game...\n");
        break;
    case 3:
        printf("Opening settings...\n");
        break;
    case 4:
        printf("Quitting...\n");
        break;
    default:
        printf("Invalid choice\n");
}

```

Example 2: switch for Menu

```

// Vowel or consonant checker using fall-through
char ch = 'a';
int isVowel = 0;

switch (ch) {
    case 'a': case 'e': case 'i': case 'o': case 'u':
    case 'A': case 'E': case 'I': case 'O': case 'U':
        isVowel = 1;
        break;
}

printf("%c is %s\n", ch, isVowel ? "vowel" : "consonant");
// Output: a is vowel

```

Example 3: Switch Fall-Through (Intentional)

Practice Questions

MCQ 1: What is the output?

```

int x = 5;
if (x = 10) {
    printf("%d", x);
}

```

- A) 5
- B) 10
- C) Nothing
- D) Error

MCQ 2: What is missing in this switch?

```
switch (x) {  
    case 1: printf("One");  
    case 2: printf("Two"); break;  
}
```

- A) break after case 1
- B) default case
- C) Parentheses around x
- D) Nothing

Short Answer: Explain why `if (x = 5)` compiles but `if (x = y)` might not be what the programmer intended.

Board Coding: Write a switch that prints the number of days in a month (ignore leap years).

3.2 Iteration Statements (Loops)

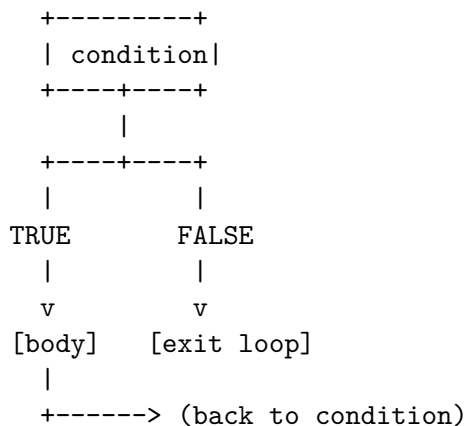
Definition / Theoretical Background

Iteration statements (loops) repeat a block of code multiple times.

The while Loop

```
while (condition) {  
    // body executed repeatedly  
    // condition checked before each iteration  
}
```

Flowchart:



Example:

```
int i = 0;  
while (i < 5) {
```

```

    printf("%d ", i); // Prints: 0 1 2 3 4
    i++;
}

```

The for Loop

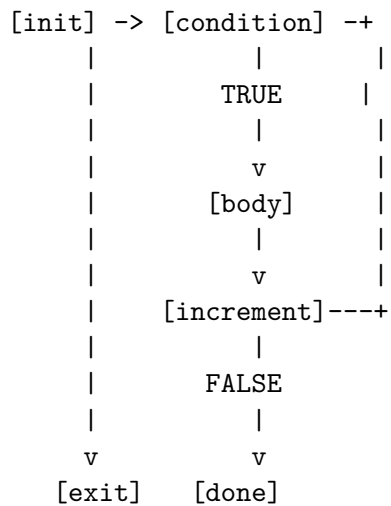
```

for (init; condition; increment) {
    // body
}

// Same as:
init;
while (condition) {
    body
    increment;
}

```

Flowchart:



Examples:

```

// Classic for loop
for (int i = 0; i < 5; i++) {
    printf("%d ", i); // 0 1 2 3 4
}

// Multiple variables
for (int i = 0, j = 10; i < j; i++, j--) {
    printf("%d %d ", i, j); // 0 10 1 9 2 8 3 7 4 6
}

// Empty parts (infinite loop)
for (;;) {
    // Infinite loop - needs break inside!
}

```

```
}
```

The do-while Loop

```
do {  
    // body executed at least once  
    // condition checked after each iteration  
} while (condition);
```

Key Difference: Body executes BEFORE condition check, so always runs at least once!

```
int i = 0;  
do {  
    printf("%d ", i);  
    i++;  
} while (i < 5); // Prints: 0 1 2 3 4  
  
// vs while (might not execute at all):  
i = 0;  
while (i < 0) {  
    printf("%d ", i); // Never prints!  
    i++;  
}
```

Loop Control: break and continue

```
for (int i = 0; i < 10; i++) {  
    if (i == 5) {  
        break; // Exit loop immediately when i is 5  
    }  
    printf("%d ", i); // Prints: 0 1 2 3 4  
}  
// Loop exits when i reaches 5
```

break - Exit Loop Immediately

```
for (int i = 0; i < 5; i++) {  
    if (i == 2) {  
        continue; // Skip rest of this iteration  
    }  
    printf("%d ", i); // Prints: 0 1 3 4 (skips 2)  
}
```

continue - Skip to Next Iteration break vs continue:

break: exits the loop entirely

continue: skips to next iteration

Edge Cases & Traps

Trap 1: Off-By-One in Loop Condition

```
// Common mistake: using <= instead of <
for (int i = 0; i <= 4; i++) { // Goes 0,1,2,3,4 (5 elements!)
    // Correct for 5 elements
}

int arr[5] = {1, 2, 3, 4, 5};
// Wrong: for (int i = 0; i <= 5; i++) - out of bounds at i=5!
// Correct: for (int i = 0; i < 5; i++)
```

[WARNING] **Professor Trap:** “Write a loop to print array elements” - students often use `i <= n` instead of `i < n`!

Trap 2: Infinite Loop (Forgot to Update)

```
int i = 0;
while (i < 5) {
    printf("%d", i);
    // BUG: forgot i++ - infinite loop!
}
```

[WARNING] **Professor Trap:** Always ensure the loop variable is updated!

Trap 3: Loop Variable Modification Inside

```
for (int i = 0; i < 5; i++) {
    printf("%d ", i);
    i = i + 1; // Modifies loop variable! Now prints: 0 2 4
    // i becomes 1, then increment makes it 2, then loop adds 1 makes it 3...
    // Actually: starts 0, prints 0, i becomes 2 (0+1+1)
    // then condition: 2 < 5, i becomes 4, prints 4...
    // Output: 0 4
}
```

Trap 4: Empty Loop Body

```
// All work in the for header
for (int i = 0; i < 1000; i++); // Oops! Semicolon ends loop immediately!

printf("Done"); // This executes, but loop body was empty
// Result: prints "Done" but did nothing useful
```

[WARNING] **Professor Trap:** A semicolon after `)` creates an empty loop body!

Trap 5: do-while Without Semicolon

```
int i = 0;
do
    printf("%d", i);
    i++;
```

```
while (i < 5); // WRONG! Only printf is part of do-while!
```

```
// Correct:
```

```
do {  
    printf("%d", i);  
    i++;  
} while (i < 5);
```

Trap 6: Comparison Bug with continue

```
for (int i = 0; i < 5; i++) {  
    if (i == 2)  
        continue; // If i is 2, skip to increment (not past the condition check!)  
    printf("%d", i);  
}  
// Prints: 0 1 3 4 (correctly skips 2)
```

Examples

```
int arr[] = {1, 2, 3, 4, 5};  
int sum = 0;  
int n = sizeof(arr) / sizeof(arr[0]);  
  
for (int i = 0; i < n; i++) {  
    sum += arr[i];  
}  
printf("Sum: %d\n", sum); // 15
```

Example 1: Summing Array Elements

```
int arr[] = {3, 7, 2, 9, 4};  
int max = arr[0];  
  
for (int i = 1; i < 5; i++) {  
    if (arr[i] > max) {  
        max = arr[i];  
    }  
}  
printf("Max: %d\n", max); // 9
```

Example 2: Finding Maximum

```
char str[] = "Hello";  
int len = strlen(str);
```

```

for (int i = len - 1; i >= 0; i--) {
    printf("%c", str[i]); // olleH
}
printf("\n");

// While version:
int i = len - 1;
while (i >= 0) {
    printf("%c", str[i]);
    i--;
}
printf("\n");

```

Example 3: Reverse String

```

// Print 3x3 multiplication table
for (int i = 1; i <= 3; i++) {
    for (int j = 1; j <= 3; j++) {
        printf("%3d ", i * j);
    }
    printf("\n");
}

/* Output:
    1  2  3
    2  4  6
    3  6  9
*/

```

Example 4: Nested Loops (Matrix)

Practice Questions

MCQ 1: How many times does this loop execute?

```

for (int i = 0; i < 5; i++) {
    printf("%d", i);
}

```

- A) 4 times
- B) 5 times
- C) 6 times
- D) Infinite

MCQ 2: What's the output?

```
int i = 0;
while (i < 3) {
    if (i == 1) continue;
    printf("%d", i);
    i++;
}
```

- A) 012
- B) 02
- C) 0 (infinite loop!)
- D) 013

MCQ 3: What is the difference between break and continue? - A) They are the same - B) break exits loop, continue skips iteration - C) continue exits, break skips - D) Neither affects loop

Short Answer: Why is do-while guaranteed to execute at least once?

Board Coding: Write a program to check if a number is prime using a for loop.

3.3 Jump Statements

Definition / Theoretical Background

Jump statements transfer control to another location in the program.

return Statement

```
// Return from function
int add(int a, int b) {
    return a + b; // Returns sum to caller
}

// Return with no value (void functions)
void printHello() {
    printf("Hello");
    return; // Optional in void functions
}
```

goto Statement (AVOID USING!)

```
// goto jumps to a label
goto label;
// ... code ...
label:
    // code after label
```

```
// Example - structured goto (still considered harmful!)
for (int i = 0; i < 10; i++) {
    for (int j = 0; j < 10; j++) {
        if (i == 5 && j == 5) {
            goto found; // Break out of nested loops
        }
    }
}
found:
printf("Found at 5,5\n");

// Why goto is discouraged:
```

[WARNING] **Professor Trap:** goto can create “spaghetti code” - tangled control flow that’s hard to follow. Use break/continue/functions instead!

When to Use goto (Rare Exceptions)

```
// Error handling pattern (rare acceptable use)
int process() {
    int *ptr = malloc(sizeof(int));
    if (!ptr) goto error1;

    FILE *f = fopen("file.txt", "r");
    if (!f) goto error2;

    // ... use file and ptr ...
    fclose(f);
    free(ptr);
    return 0;

error2:
    free(ptr);
error1:
    return -1;
}
```

Comparison of Loop Exit Methods

Method	What It Does	Scope
break	Exits innermost loop	Loop only
continue	Skips to next iteration	Loop only
return	Exits function entirely	Function
goto	Jumps to label	Anywhere in function

Edge Cases & Traps

Trap 1: Multiple returns in function

```
int max(int a, int b) {  
    if (a > b) return a; // Early return  
    return b;           // Late return  
}  
  
// Both returns are valid, but don't use both unnecessarily
```

Trap 2: Return without value

```
void foo() {  
    return 5; // ERROR! Can't return value from void function!  
}  
  
int bar() {  
    // Missing return - may cause warning or undefined behavior  
}
```

Trap 3: Returning local variable address

```
int *wrong() {  
    int x = 10;  
    return &x; // WARNING! x is stack-allocated and destroyed on return  
}
```

Practice Questions

MCQ 1: Which statement can exit a function from within a loop? - A) break - B) continue - C) return - D) goto and return

Short Answer: Why is goto generally discouraged in modern programming?

3.4 Pattern Matching in Control Flow

Common Control Flow Patterns

```
// Without guard clause  
void processData(int *data, int size) {  
    if (data != NULL) {  
        if (size > 0) {  
            if (size < 1000) {  
                // Actual work here  
            }  
        }  
    }  
}
```

```
// With guard clauses (better!)
void processData(int *data, int size) {
    if (data == NULL) return;           // Guard: exit early
    if (size <= 0) return;              // Guard: exit early
    if (size >= 1000) return;          // Guard: exit early
    // Actual work here
}
```

Early Exit / Guard Clauses

```
enum State { IDLE, RUNNING, PAUSED, STOPPED };
State current = IDLE;

while (1) {
    switch (current) {
        case IDLE:
            if (startPressed) current = RUNNING;
            break;
        case RUNNING:
            if (pausePressed) current = PAUSED;
            if (stopPressed) current = STOPPED;
            break;
        case PAUSED:
            if (resumePressed) current = RUNNING;
            if (stopPressed) current = STOPPED;
            break;
        case STOPPED:
            return; // Exit program
    }
}
```

State Machine Pattern

```
while (1) {
    printMenu();
    int choice = getChoice();

    switch (choice) {
        case 1: doOption1(); break;
        case 2: doOption2(); break;
        case 3: doOption3(); break;
        case 4: printf("Goodbye"); return; // Exit program
        default: printf("Invalid choice");
    }
}
```

Menu-Driven Program

Cheat-Sheet

Statement	Purpose	Key Feature
<code>if</code>	Conditional execution	Checks condition first
<code>switch</code>	Multi-way branch	Integer/char only, fall-through
<code>while</code>	Pre-test loop	May execute 0 times
<code>for</code>	Counter loop	init;cond;incr in one line
<code>do-while</code>	Post-test loop	Always executes at least once
<code>break</code>	Exit loop/switch	Innermost only
<code>continue</code>	Next iteration	Skips rest of body
<code>return</code>	Exit function	Returns value (optional)
<code>goto</code>	Jump anywhere	AVOID - creates spaghetti code

Review Checklist

- ☐ **if/else:** Can you write if-else chains correctly?
- ☐ **switch:** Do you understand fall-through and when to use it?
- ☐ **Loops:** Can you write while, for, and do-while loops?
- ☐ **break/continue:** Do you know the difference?
- ☐ **Common bugs:** Off-by-one, assignment in condition, missing break?
- ☐ **Control flow patterns:** Can you identify guard clauses, state machines? # Chapter 4: Types

4.1 Basic Types and Their Representation

Definition / Theoretical Background

Types define the set of values a variable can hold and the operations that can be performed on it. Understanding how types are represented in memory is crucial for writing efficient, bug-free code.

Integer Types

Common Integer Types and Sizes (Typical on 64-bit systems)

Type	Size	Range	Example
<code>char</code>	1 byte	-128 to 127 (signed) or 0 to 255 (unsigned)	<code>char c = 'A';</code>
<code>short</code>	2 bytes	-32,768 to 32,767	<code>short s = 1000;</code>
<code>int</code>	4 bytes	-2,147,483,648 to 2,147,483,647	<code>int x = 42;</code>
<code>long</code>	4 or 8 bytes	Depends on system	<code>long l = 100000L;</code>
<code>long long</code>	8 bytes	-9.2x10 ¹⁸ to 9.2x10 ¹⁸	<code>long long ll = 1e18;</code>

Integer in Memory: Two's Complement Representation Why Two's Complement? - One representation for zero - Arithmetic operations work uniformly - Range: $-2^{(n-1)}$ to $2^{(n-1)} - 1$

Decimal	Binary	Notes
0	-> 0000	
1	-> 0001	
2	-> 0010	
3	-> 0011	
7	-> 0111	Maximum positive ($2^3 - 1$)
-1	-> 1111	1111 + 0001 = 1 0000 (carry discarded)
-2	-> 1110	1110 + 0010 = 1 0000
-8	-> 1000	Minimum negative (-2^3)

```
Step 1: Take absolute value: 5
Step 2: Convert to binary: 0101
Step 3: Invert all bits: 1010
Step 4: Add 1: 1010 + 1 = 1011
```

Step-by-Step: Converting 1011 (Two's Complement) to Decimal:

Important Range Calculation:

Min: $-2^{(n-1)}$
Max: $2^{(n-1)} - 1$

Min: $-2^7 = -128$
Max: $2^7 - 1 = 127$

IEEE 754 Single Precision (float) - 32 bits



Formula: $(-1)^S \times 2^{(E-127)} \times (1 + M/2^{23})$

64-bit Double		
1 bit	11 bits	52 bits
Sign	Exponent	Mantissa (Significand)
S	E (biased)	M
$2^{(E-1023)}$		$1 + M/2^{52}$ (normalized)

Formula: $(-1)^S \times 2^{(E-1023)} \times (1 + M/2^{52})$

Float Memory Layout Example

59

Special Float Values

```
# Single precision (float) vs Double precision (double)
# float: 23-bit mantissa ~ 7 decimal digits
# double: 52-bit mantissa ~ 15 decimal digits

# Python float is 64-bit (double)
result = 0.1 + 0.2
print(result) # 0.30000000000000004

# C float (32-bit) has even less precision
float a = 0.1f;
float b = 0.2f;
float c = a + b;
printf("%.20f\n", c); // 0.300000011920928955... (less precise!)
```

Why Float Precision Matters

Edge Cases & Traps

Trap 1: Integer Overflow

```
// UNDEFINED BEHAVIOR in C!
int x = INT_MAX; // 2,147,483,647
x = x + 1; // OVERFLOW! Could be negative, could be anything!

// What happens: 2,147,483,647 + 1 = -2,147,483,648 in two's complement
// But this is actually undefined behavior - compiler may optimize away checks!
```

[WARNING] **Professor Trap:** Integer overflow is UNDEFINED for SIGNED integers! Don't rely on wrapping!

Trap 2: Unsigned Overflow (Defined but surprising)

```
// Defined behavior for UNSIGNED integers (wraps around)
unsigned int y = UINT_MAX; // 4,294,967,295
y = y + 1; // y = 0 (wraps around, this is DEFINED!)

// But be careful:
unsigned int z = -1; // -1 converted to unsigned = 4,294,967,295!
printf("%u", z); // Prints 4294967295
```

Trap 3: Float Precision with Large Numbers

```
# What happens here?
large = 16777217.0
print(large == 16777216.0 + 1) # True? False?
# Answer: May be FALSE in some float representations!
```

```
# 16,777,217 cannot be represented exactly as float!  
# Closest float: 16777216.0 (single precision can't distinguish)
```

[WARNING] **Professor Trap:** Not all integers can be exactly represented as floats!

Trap 4: Integer vs Float Division

```
// Integer division truncates!  
int a = 5, b = 2;  
printf("%d", a / b);    // 2 (not 2.5!)  
printf("%f", (float)a / b); // 2.500000  
  
// Even with floats on the left, if b is int, might still truncate!  
float result = a / b; // 2.000000! BOTH are integers, integer division first!  
result = (float)a / (float)b; // 2.500000 - correct!
```

Trap 5: sizeof() Returns Size in Bytes

```
// sizeof gives bytes, NOT bits!  
int arr[10];  
printf("%zu", sizeof(arr)); // 40 (bytes), NOT 320 (bits)!  
  
// To get bits:  
int bits = sizeof(arr) * 8; // 40 * 8 = 320 bits
```

Trap 6: Implicit Conversion to Signed/Unsigned

```
int x = -10;  
unsigned int y = 5;  
  
int result = x + y; // What is this?  
  
// Answer: x is converted to unsigned!  
// -10 as unsigned = 4,294,967,286  
// result = 4,294,967,286 + 5 = 4,294,967,291 (huge positive!)  
// This is UNDEFINED if result is negative after conversion!
```

[WARNING] **Professor Trap:** Mixing signed and unsigned can give surprising results!

Trap 7: Float Comparison

```
// NEVER compare floats with ==  
double a = 0.1 * 3; // 0.30000000000000004  
double b = 0.3;  
  
if (a == b) { // ALWAYS FALSE!  
    printf("Equal");  
} else {  
    printf("Not equal: a=%.20f, b=%.20f", a, b);  
    // Output: Not equal: a=0.30000000000000004, b=0.30000000000000000  
}
```

```

// CORRECT ways to compare floats:
if (fabs(a - b) < 1e-9) { // Within tolerance
    printf("Approximately equal");
}

#include <math.h>
if (isclose(a, b, rel_tol=1e-9, abs_tol=0.0)) { // C11
    printf("Close enough");
}

```

Examples

```

// Show memory representation of an integer
#include <stdio.h>

int main() {
    int x = 1; // 0x00000001 in memory (little-endian)
    unsigned int y = 0x41424344; // 'ABCD' as int

    // Print each byte individually (assuming little-endian)
    unsigned char *p = (unsigned char*)&x;
    printf("int %d bytes: ", (int)sizeof(int));
    for (int i = 0; i < sizeof(int); i++) {
        printf("%02X ", p[i]);
    }
    printf("\n");
    // Output: int 4 bytes: 01 00 00 00 (little-endian: least significant byte first)

    // For 0x41424344:
    // 44 43 42 41 in memory (reads as 'DCBA' in little-endian)
    char *s = (char*)&y;
    printf("String bytes: %c%c%c%c\n", s[0], s[1], s[2], s[3]);
    // Output: DCBA (byte order reversed)

    return 0;
}

```

Example 1: Integer Representation in Memory

```

#include <stdio.h>

void print_float_bits(float f) {
    unsigned int bits = *(unsigned int*)&f;

    printf("Float %.2f = 0x%08X\n", f, bits);
}

```

```

    // Extract components
    unsigned int sign = (bits >> 31) & 1;
    unsigned int exp = (bits >> 23) & 0xFF;
    unsigned int mantissa = bits & 0x7FFFFFFF;

    printf("  Sign: %u\n", sign);
    printf("  Exponent: %u (bias=%u, actual=%d)\n", exp, 127, (int)exp - 127);
    printf("  Mantissa: 0x%06X (%.10f)\n", mantissa, 1.0 + mantissa / (float)(1 << 23));
}

int main() {
    print_float_bits(1.0f);    // 0x3F800000
    print_float_bits(-1.0f);   // 0xBF800000
    print_float_bits(0.5f);    // 0x3F000000
    print_float_bits(0.1f);    // 0x3DCCCCCD (NOT 0.1 exactly!)

    return 0;
}

```

Example 2: Float Bit Pattern Analysis

```

#include <limits.h>
#include <stdio.h>

int main() {
    printf("Integer Limits:\n");
    printf("  int: %d to %d\n", INT_MIN, INT_MAX);
    printf("  unsigned int: 0 to %u\n", UINT_MAX);

    // Check for overflow
    int x = INT_MAX;
    printf("INT_MAX + 1 = %d\n", x + 1); // Undefined!

    // Safe unsigned overflow
    unsigned int y = UINT_MAX;
    printf("UINT_MAX + 1 = %u\n", y + 1); // Defined: wraps to 0

    // Float limits
    printf("Float: %.10e\n", FLT_MAX); // ~3.4e38
    printf("Float min positive: %.10e\n", FLT_MIN); // ~1.2e-38

    return 0;
}

```

Example 3: Range Limits

Practice Questions

MCQ 1: How many values can a 3-bit unsigned integer represent? - A) 4 - B) 6 - C) 7 - D) 8

MCQ 2: What is the two's complement representation of -3 in 4 bits? - A) 1101 - B) 1110 - C) 1011 - D) 1100

MCQ 3: In IEEE 754 single precision, what is the bias value? - A) 127 - B) 128 - C) 1023 - D) 23

Short Answer: Explain why $0.1 + 0.2 \neq 0.3$ in floating-point.

Board Coding: Show step-by-step how -7 is represented in 8-bit two's complement.

Oral Explanation: Explain the difference between float (32-bit) and double (64-bit) in terms of range and precision.

Solutions

MCQ 1: D) 8 (0 to 7 = 8 values, formula: 2^n)

MCQ 2: A) 1101

7 in binary = 0111

Invert = 1000

Add 1 = 1001

Wait, let me recalculate:

7 = 0111

Invert = 1000

Add 1 = 1001

-7 = 1001 in two's complement

-3:

3 = 0011

Invert = 1100

Add 1 = 1101

Answer: 1101 [OK]

MCQ 3: A) 127 (for single precision; double uses 1023)

Short Answer: 0.1 and 0.2 cannot be represented exactly in binary floating-point. They are repeating fractions. When added, $0.1 + 0.2 = 0.30000000000000004$, which is slightly larger than 0.3. Always use tolerance-based comparison for floats. # Chapter 4 Part 2: Type Conversions

4.2 Automatic and Explicit Type Conversions

Definition / Theoretical Background

Type conversion (also called **type casting**) is the process of converting a value from one type to another.

Types of Type Conversion

Type	Also Called	What Happens
Implicit	Coercion, automatic	Compiler automatically converts
Explicit	Casting	Programmer specifies conversion

Implicit Type Conversion (Automatic)

Also known as: Type coercion, type promotion

The compiler automatically converts types in certain situations:

Conversion Hierarchy (usually in this order):

```
char -> short -> int -> long -> float -> double
      ^
      |
      signed <-> unsigned (complex!)
```

Common Implicit Conversion Scenarios Scenario 1: Assignment with different types

```
int x = 3.14;      // x = 3 (fractional part truncated!)
float y = 10;      // y = 10.0 (int converted to float)
char c = 65;       // c = 'A' (65 is ASCII for 'A')
```

Scenario 2: Expressions with mixed types

```
int a = 5, b = 2;
float result = a / b; // result = 2.0, NOT 2.5!
// Why? Both a and b are int, so integer division first, THEN assign to float

float correct = (float)a / b; // correct = 2.5
```

Scenario 3: Function arguments

```
void printDouble(double d);

int x = 5;
printDouble(x); // x automatically converted to double
```

The JavaScript Type Coercion Chaos

JavaScript's automatic type conversion is notorious and produces surprising results.

```
// JavaScript coercion - BE CAREFUL!
console.log("1" + 1);      // "11" (string concatenation!)
console.log("5" - 1);      // 4 (numeric subtraction!)
console.log("5" == 5);     // true (loose equality, coerced!)
console.log("5" === 5);    // false (strict equality, no coercion!)
console.log(true + "1");   // "true1" (true becomes "true")
console.log(false - "1");  // -1 (false becomes 0)
```

```

console.log(null == 0);    // true? No! null coerces to 0 in == but not ===
console.log([] + {});     // "[object Object]"
console.log({} + []);     // 0 ( {} is block, +[] is 0)

```

The Famous “1” + 1 = 11 Problem

JavaScript Type Coercion Rules Table

Expression	Result	Explanation
"5" + 1	"51"	String concatenation
"5" - 1	4	Numeric subtraction
"5" * "2"	10	Numeric multiplication
"hello" * 1	NaN	Not a Number
true + 1	2	true -> 1
false + 1	1	false -> 0
null + 1	1	null -> 0
undefined + 1	NaN	undefined -> NaN
[] + {}	"[object Object]"	Array empty string + object
{ } + []	0	Empty block + numeric [] = 0
" " == 0	true	Space string -> 0
"" == 0	true	Empty string -> 0

```

// Step 1: Operator is + (can be addition OR concatenation)
// Step 2: Check if either operand is a string
// Step 3: If yes, convert the other to string and concatenate
// Step 4: If no, perform numeric addition

```

```

"1" + 1
  ^

```

This is a string!

```

// Therefore: convert 1 to "1", then concatenate -> "11"

```

Deep Dive: Why “1” + 1 = “11”

```

// Step 1: Operator is - (subtraction, NOT string operation)
// Step 2: No string involved, so numeric subtraction
// Step 3: Convert both to numbers, then subtract
// Step 4: "5" becomes 5, 1 stays 1, result = 4

```

```

"5" - 1
  ^

```

Minus is only numeric, no string concatenation!

Deep Dive: Why “5” - 1 = 4

```
// == (loose equality) - COERCES types!
console.log("5" == 5);    // true (string "5" becomes number 5)
console.log(0 == false);  // true (false becomes 0)
console.log("" == 0);     // true (empty string becomes 0)
console.log(null == 0);   // true? Actually FALSE!
console.log(undefined == 0); // false! undefined becomes NaN!

// === (strict equality) - NO coercion, compares types AND values
console.log("5" === 5);   // false (different types!)
console.log(0 === false); // false (different types!)
console.log("" === 0);    // false (different types!)

// RECOMMENDATION: ALWAYS use === unless you specifically want coercion!
```

The == vs === Trap (CRITICAL!)

```
// The infamous empty object + empty array
console.log({} + []); // Some browsers: "[object Object]"
                    // Some browsers: 0

// Why? Depends on how the engine parses it!
// {} might be a block or object literal
// +[] could be unary plus converting empty array to number

console.log([] + {}); // "[object Object]"
// [] becomes "", {} becomes "[object Object]", concatenate

// The real lesson: Don't rely on these coercions!
```

Empty String and Zero Weirdness

Explicit Type Conversion (Casting)

```
// Syntax: (type)value
int x = 10;
float y = (float)x;    // Explicit cast

// Common casts
char c = (char)65;     // c = 'A'
double d = (double)10; // d = 10.0
int *ptr = (int*)0x1234; // Pointer from integer constant
```

C Syntax (Type Enclosed in Pareds)

```
int x = 10;
float y = float(x);           // Function-style cast
float z = static_cast<float>(x); // C++ style - preferred!
```

C++ Syntax (Function-Style and static_cast)

```
// To String
String(123);           // "123"
(123).toString();      // "123"
123 + "";              // "123" (implicit but explicit intent)

// To Number
Number("123");         // 123
parseInt("123", 10);    // 123 (parses until non-digit)
parseFloat("3.14");     // 3.14
+"123";                // 123 (unary plus)

// To Boolean
Boolean(x);            // true/false
!!x;                   // Double negation, explicit
```

JavaScript Explicit Conversion

Edge Cases & Traps

Trap 1: JavaScript Addition vs String Concatenation

```
// What is the result?
let result = "1" + 2 + 3; // "123" or "15"?
// Actually: "123" - left to right
// "1" + 2 = "12"
// "12" + 3 = "123"

let result2 = 1 + 2 + "3"; // "33" or "33"?
// 1 + 2 = 3
// 3 + "3" = "33"

let result3 = 1 + 2 + 3; // 6 (pure arithmetic)
```

[WARNING] Professor Trap: "1" + 2 + 3 = "123", but 1 + 2 + "3" = "33"!

Trap 2: JavaScript Loose Equality == Weirdness

```
// These are ALL true with == but false with ===:
console.log("" == 0);           // true (empty string -> 0)
console.log(false == 0);        // true (false -> 0)
console.log("" == false);       // true (both become 0!)
console.log(null == undefined); // true (special case)
```

```
console.log("\n" == 0);      // true (whitespace -> 0)

// But:
console.log(null === undefined); // false (different types!)
```

[WARNING] **Professor Trap:** NEVER use == unless you understand coercion fully!

Trap 3: C Integer Promotion in Expressions

```
// What is the size of x?
char a = 50, b = 50;
int x = a * b; // What is sizeof(x)? 4 bytes!

// Why? a and b are char, but they're promoted to int for multiplication!
// a * b = 2500 (as int)
// x = 2500 (as int)
```

Trap 4: Implicit Narrowing in C

```
// This compiles but may lose data!
int big = 257;
char small = big; // small = 1 (only lower 8 bits kept!)
printf("%d", small); // 1

// float to int truncates!
float f = 3.99;
int i = f; // i = 3 (fractional part lost!)
```

Trap 5: Converting Negative to Unsigned

```
// What happens?
int x = -1;
unsigned int y = x;
printf("%u", y); // 4294967295 (on 32-bit!)

// Explanation: -1 in two's complement = all 1s
// When interpreted as unsigned, = UINT_MAX
```

[WARNING] **Professor Trap:** Converting negative to unsigned gives unexpected results!

Trap 6: sizeof with Cast

```
// What does this print?
int x = 5;
printf("%zu", sizeof((float)x)); // 4 - the cast doesn't change size!
printf("%zu", sizeof(x));        // 4 - x is still an int!

// sizeof is computed at COMPILE TIME based on type, not value!
```

Examples

```

// The classic trap: integer division
#include <stdio.h>

int main() {
    int a = 5, b = 2;
    float f1 = a / b;           // WRONG: 2.000000
    float f2 = (float)a / b;    // RIGHT: 2.500000
    float f3 = a / (float)b;    // RIGHT: 2.500000
    float f4 = (float)a / (float)b; // RIGHT: 2.500000

    printf("a / b (ints):  %.6f\n", f1);
    printf("(float)a / b:  %.6f\n", f2);
    printf("a / (float)b:  %.6f\n", f3);
    printf("(float)a / b:  %.6f\n", f4);

    return 0;
}

```

Example 1: C Integer Division Truncation

```

// What gets printed?
if ("" ) {
    console.log("Empty string is truthy");
} else {
    console.log("Empty string is falsy"); // THIS!
}

if ([]) {
    console.log("Empty array is truthy"); // THIS!
} else {
    console.log("Empty array is falsy");
}

// Falsy values in JavaScript:
console.log(Boolean(false)); // false
console.log(Boolean(0));     // false
console.log(Boolean(""));   // false
console.log(Boolean(null));  // false
console.log(Boolean(undefined)); // false
console.log(Boolean(NaN));   // false

// Everything else is truthy, including:
console.log(Boolean("false")); // true (non-empty string!)
console.log(Boolean([]));      // true (empty array!)
console.log(Boolean({}));      // true (empty object!)

```

Example 2: JavaScript Coercion in Conditionals

```
#include <stdio.h>

void printSize(char c) {
    printf("char promoted to int: %zu\n", sizeof(c));
}

int main() {
    char c = 'A';
    printf("Size of char: %zu\n", sizeof(c));          // 1
    printf("Size of c in func: %zu\n", sizeof(c));    // 4 (promoted!)

    // But actual char is passed - no conversion!
    printSize(c);
    return 0;
}
```

Example 3: Type Promotion in Function Calls

```
#include <stdio.h>

int main() {
    // Safe: within range
    int big = 100;
    char small = (char)big; // Safe, within char range

    // Unsafe: overflow
    int tooBig = 300;
    char overflow = (char)tooBig; // overflow! small = 44 (300 % 256)

    printf("100 as char: %d\n", (char)100); // 100
    printf("300 as char: %d\n", (char)300); // 44 (wrapped!)

    // Checking before cast
    int value = 1000;
    if (value >= -128 && value <= 127) {
        char safe = (char)value;
        printf("Safe cast: %d\n", safe);
    } else {
        printf("Value %d out of char range!\n", value);
    }

    return 0;
}
```

Example 4: Safe Type Conversion

Practice Questions

MCQ 1: In JavaScript, what is "5" + 2? - A) 7 - B) "52" - C) 52 - D) Error

MCQ 2: What is the result of 5 / 2 in C when both operands are integers? - A) 2.5 - B) 2 - C) 2.0 - D) Error

MCQ 3: In C, what happens when you assign a float to an int variable? - A) Compile error - B) Rounds to nearest integer - C) Truncates fractional part - D) Undefined

Short Answer: Explain why "5" == 5 is true in JavaScript but "5" === 5 is false.

Board Coding: Write C code that safely converts an integer to a char, checking for overflow first.

Oral Explanation: Explain the difference between implicit and explicit type conversion. When is each appropriate?

Solutions

MCQ 1: B) "52" – string concatenation

MCQ 2: B) 2 – integer division truncates

MCQ 3: C) Truncates fractional part

Short Answer: The == operator in JavaScript performs type coercion, converting "5" to number 5 before comparing. The === operator (strict equality) does NOT coerce, so it compares a string to a number and returns false because types differ.

Board Coding:

```
int value = 1000;
char safe;

// Check range before casting
if (value >= -128 && value <= 127) {
    safe = (char)value;
    printf("Safe: %d\n", safe);
} else {
    printf("Overflow would occur!\n");
}
```

Chapter 4 Part 3: Arrays

4.3 Compound Types: Arrays

Definition / Theoretical Background

An **array** is a contiguous block of memory containing elements of the same type, accessed by integer indices. Unlike a list, arrays have fixed size and direct ($O(1)$) access to any element.

Key Properties of Arrays: - **Fixed size** (in most languages) - size must be known at compile time or allocated explicitly - **Contiguous memory** - all elements stored side-by-side - **Homogeneous** - all elements are the same type - **Direct access** - any element accessed in constant time via index

Array vs List: The Critical Difference

This is a VERY COMMON EXAM QUESTION!

Property	Array	List (Linked List)
Memory	Contiguous block	Scattered (nodes connected by pointers)
Size	Fixed (static) or sized at allocation	Dynamic, grows as needed
Access	O(1) direct by index	O(n) must traverse from head
Insert/Delete	O(n) (shifting elements)	O(1) if at known position
Memory overhead	Low (just data)	Higher (need next/prev pointers)
Cache locality	Excellent (contiguous)	Poor (scattered)
Example	<code>int arr[5];</code> in C	<code>ArrayList<Integer></code> in Java

ARRAY (contiguous memory):

```
+-----+-----+-----+-----+-----+
| 10  | 20  | 30  | 40  | 50  |  <- stored side-by-side
+-----+-----+-----+-----+-----+
    0      1      2      3      4      <- indices
```

LIST (scattered memory):

```
+-----+      +-----+      +-----+
| 10  | |--->| 20  | |--->| 30  |
+-----+      +-----+      +-----+
node 1      node 2      node 3
(at addr    (at addr    (at addr
0x1000)      0x2000)      0x3000)
```

Why Arrays Have O(1) Access:

```
// In C: arr[i] = *(arr + i)
// Address of arr[i] = base_address + (i * element_size)
// ONE multiplication and ONE addition = constant time!

int arr[5] = {10, 20, 30, 40, 50};
// If arr is at address 0x1000:
// arr[0] at 0x1000 + (0 * 4) = 0x1000
// arr[3] at 0x1000 + (3 * 4) = 0x100C - just one calculation!
```

Why Lists Have O(n) Access:

```
# Python list is actually an array (dynamic array), not linked list!
# True linked list example:
class Node:
    def __init__(self, data):
        self.data = data
```

```

        self.next = None

# To find element at index 3:
# Start at head -> node1 -> node2 -> node3 -> found
# Must visit 3 nodes = O(n)

```

1D Arrays

```

// C: Declaration
int arr[5];           // Uninitialized (garbage values)
int arr[5] = {0};     // All elements = 0
int arr[5] = {1, 2, 3, 4, 5}; // Explicit initialization
int arr[] = {1, 2, 3}; // Compiler infers size = 3

// Memory layout (assuming 4-byte int, little-endian):
// Address: 0x1000 0x1004 0x1008 0x100C 0x1010
// Value:   arr[0] arr[1] arr[2] arr[3] arr[4]
//          [ 1 ] [ 2 ] [ 3 ] [ 4 ] [ 5 ]

```

Declaration and Memory Layout

```

int arr[5] = {10, 20, 30, 40, 50};

// arr is the address of the first element (pointer to arr[0])
printf("%p\n", arr);           // Address of arr[0]
printf("%p\n", &arr[0]);      // Same address!

// arr[i] is same as *(arr + i)
printf("%d\n", arr[2]);        // 30
printf("%d\n", *(arr + 2));    // 30 (same!)
printf("%d\n", 2[arr]);        // 30 (valid! arr[2] == *(arr+2) == *(2+arr) == 2[arr])

```

Indexing and Pointers

```

int arr[5] = {1, 2, 3, 4, 5};
// Valid indices: 0, 1, 2, 3, 4

printf("%d", arr[5]);          // DANGER! Out of bounds - accessing arr[5] doesn't exist!
printf("%d", arr[-1]);         // DANGER! Negative index - undefined behavior!

// C has NO bounds checking by default!
// arr[5] might give you garbage, crash, or anything!

```

Array Bounds (CRITICAL!)

2D Arrays

Memory Layout: Row-Major vs Column-Major C uses **Row-Major** (row by row, then column by column)

```
int matrix[2][3] = {
    {1, 2, 3},    // Row 0
    {4, 5, 6}     // Row 1
};

// Row-Major Memory Layout (in C):
// [1] [2] [3] [4] [5] [6]
// 0  1  2  3  4  5  <- Linear indices
// Row0    Row1

// Address calculation in row-major:
// &matrix[row][col] = base + (row * numCols + col) * elementSize
```

Visual representation:

```
Matrix (2 rows, 3 columns):
+-----+
| matrix[0][0] | 1 | 2 | 3 | Row 0
+-----+
| matrix[1][0] | 4 | 5 | 6 | Row 1
+-----+
```

```
Memory (row by row, left to right):
+---+---+---+---+---+
| 1 | 2 | 3 | 4 | 5 | 6 |
+---+---+---+---+---+
0  1  2  3  4  5  (linear index)
```

FORTRAN uses **Column-Major** (column by column, then row by row)

```
! FORTRAN (column-major)
integer :: mat(2,3) = reshape([1,2,3,4,5,6], [2,3])

! Memory layout: [1] [4] [2] [5] [3] [6]
! Same values, different order!
```

```
int matrix[2][3] = {{1, 2, 3}, {4, 5, 6}};

// Access elements
printf("%d", matrix[0][2]); // 3 (row 0, col 2)
printf("%d", matrix[1][0]); // 4 (row 1, col 0)
printf("%d", matrix[1][2]); // 6 (row 1, col 2)

// As pointer arithmetic
```

```
int *ptr = &matrix[0][0];
printf("%d", *(ptr + 5));    // 6 (ptr + 5 = element at linear index 5)
```

Accessing 2D Arrays

Array Decay (IMPORTANT!)

KEY CONCEPT: When an array is passed to a function, it “decays” to a pointer!

```
void func(int arr[]) {
    // arr is NOT an array here!
    // It's a POINTER to the first element!
    printf("%zu", sizeof(arr)); // Prints 8 (or 4) - size of POINTER, not array!
}

// In caller:
int arr[5] = {1, 2, 3, 4, 5};
printf("%zu", sizeof(arr)); // Prints 20 (5 * 4 bytes) - actual array size!

func(arr); // Array decays to int* when passed!
```

Why This Matters:

```
void modify(int arr[]) {
    arr[0] = 100; // This DOES modify the original!
    // Because arr is a pointer to the original array
}

int main() {
    int arr[5] = {1, 2, 3, 4, 5};
    modify(arr);
    printf("%d", arr[0]); // Prints 100!
}
```

The Danger of Size Loss:

```
void processArray(int arr[]) {
    // arr has "lost" its size information!
    // Cannot use sizeof(arr) to get element count!
    int n = ???; // How do we know the size?
}

// Solution 1: Pass size explicitly
void processArray(int arr[], int n) {
    for (int i = 0; i < n; i++) {
        arr[i] *= 2;
    }
}

// Solution 2: Use struct to hold array + size
```

```
typedef struct {
    int *data;
    size_t size;
} IntArray;
```

Edge Cases & Traps

Trap 1: Off-By-One Error (MOST COMMON!)

```
int arr[5] = {1, 2, 3, 4, 5};
// Valid indices: 0, 1, 2, 3, 4

for (int i = 1; i <= 5; i++) { // WRONG! i goes 1,2,3,4,5
    printf("%d ", arr[i]);     // arr[5] is out of bounds!
}

// CORRECT:
for (int i = 0; i < 5; i++) { // i goes 0,1,2,3,4
    printf("%d ", arr[i]);
}
```

[WARNING] **Professor Trap:** “Write a loop to initialize all elements” - students often use `i <= n` instead of `i < n`!

Trap 2: Array Size in sizeof (After Decay)

```
void printSize(int arr[]) {
    // WRONG: This is not the array, it's a pointer!
    printf("%zu", sizeof(arr)); // Size of pointer, not array!
}

int main() {
    int arr[10];
    printf("%zu", sizeof(arr)); // 40 (10 * 4 bytes)
    printSize(arr);             // 8 or 4 (pointer size)
}
```

[WARNING] **Professor Trap:** `sizeof` in a function receiving array parameter gives pointer size, not array size!

Trap 3: Initializing Partial Array

```
int arr[5] = {1, 2}; // What are arr[2], arr[3], arr[4]?
// arr[0] = 1, arr[1] = 2, arr[2] = 0, arr[3] = 0, arr[4] = 0
// Remaining elements are zero-initialized!

int arr2[5] = {0}; // ALL elements = 0
int arr3[5];       // GARBAGE values (uninitialized!)
```

Trap 4: Comparing Arrays with ==

```

int a[5] = {1, 2, 3, 4, 5};
int b[5] = {1, 2, 3, 4, 5};

// WRONG:
if (a == b) { // This compares ADDRESSES, not contents!
    printf("Equal"); // Will NEVER print (different addresses)
} else {
    printf("Not equal"); // ALWAYS prints this!
}

// CORRECT: Manual comparison
int equal = 1;
for (int i = 0; i < 5; i++) {
    if (a[i] != b[i]) {
        equal = 0;
        break;
    }
}

```

Trap 5: 2D Array Parameter Confusion

```

// What is the correct function signature?
void process(int matrix[2][3]) { } // Works, but limited
void process(int matrix[][3]) { } // Works, compiler knows inner dimension
void process(int matrix[2][] ) { } // ERROR! Must know column size!

// Why? Because address calculation needs to know row width:
// matrix[row][col] = *(matrix + row * 3 + col) - needs to know 3!

```

Trap 6: Variable Length Arrays (VLA) - C99

```

// C99 introduced VLAs (size determined at runtime)
int n;
scanf("%d", &n);
int arr[n]; // Valid C99, but...

// But VLAs cannot be initialized at declaration:
int arr[n] = {0}; // ERROR in C99!

// Also, VLAs are optional in C11 (may not be supported)

```

Trap 7: Stack Overflow with Large Arrays

```

// This might crash! Large arrays go on stack.
int big[1000000]; // 4MB on stack - might overflow!

// Use dynamic allocation for large arrays:
int *big = malloc(1000000 * sizeof(int)); // Goes on heap
// Don't forget: free(big);

```

Examples

```
#include <stdio.h>

void printArrayInfo(int arr[], int n) {
    printf("Array address: %p\n", (void*)arr);
    printf("Element size: %zu bytes\n", sizeof(int));
    printf("\nMemory layout:\n");

    for (int i = 0; i < n; i++) {
        printf("  arr[%d] at %p = %d\n",
            i, (void*)(arr + i), arr[i]);
    }
}

int main() {
    int arr[5] = {10, 20, 30, 40, 50};
    printArrayInfo(arr, 5);

    // Output will show:
    // arr[0] at 0x7fff... = 10
    // arr[1] at 0x7fff...+4 = 20
    // arr[2] at 0x7fff...+8 = 30
    // Each element is 4 bytes apart!

    return 0;
}
```

Example 1: Array Memory Layout Visualization

```
#include <stdio.h>

int main() {
    int matrix[2][3] = {{1, 2, 3}, {4, 5, 6}};

    // Treat as 1D array
    int *ptr = &matrix[0][0];

    printf("Linear view:\n");
    for (int i = 0; i < 6; i++) {
        printf("  ptr[%d] = %d (at %p)\n", i, ptr[i], (void*)(ptr + i));
    }

    // All 6 elements are contiguous in memory:
    // 1, 2, 3, 4, 5, 6
    // This proves 2D arrays are stored row-by-row in C!
}
```

```
    return 0;
}
```

Example 2: 2D Array as Contiguous Memory

```
#include <stdio.h>

void func(int arr[]) {
    printf("In func:\n");
    printf("    sizeof(arr) = %zu (pointer size!)\n", sizeof(arr)); // 8 or 4
    printf("    arr = %p\n", (void*)arr);
}

int main() {
    int arr[5] = {1, 2, 3, 4, 5};

    printf("In main:\n");
    printf("    sizeof(arr) = %zu (array size!)\n", sizeof(arr)); // 20
    printf("    arr = %p\n", (void*)arr);

    func(arr);

    // Same address printed - array decays to pointer!
    return 0;
}
```

Example 3: Array Decay Demonstration

```
// Problem: Does subarray exist in array?
// Return index where subarray starts, or -1 if not found

int findSubarray(int arr[], int n, int sub[], int m) {
    for (int i = 0; i <= n - m; i++) { // Can only start at 0..n-m
        int found = 1;
        for (int j = 0; j < m; j++) {
            if (arr[i + j] != sub[j]) {
                found = 0;
                break;
            }
        }
        if (found) return i;
    }
    return -1;
}
```

```
// Example:
// arr = [1, 2, 3, 4, 5], sub = [3, 4]
// Result: 2 (arr[2] = 3, arr[3] = 4)
```

Example 4: Finding Subarray in Array (Exam Question)

White-Board Walkthrough: Array Index to Address

Question: Given `int arr[5]` starting at `0x1000`, what is `&arr[3]`?

Answer: `0x1000 + 3 * sizeof(int) = 0x1000 + 12 = 0x100C`

Step-by-step:

1. `arr` is address `0x1000`
2. `arr[i]` is at: `arr + i * element_size`
3. `sizeof(int) = 4` bytes (typical)
4. `0x1000 + (3 * 4) = 0x1000 + 12 = 0x100C`

Visual:

Memory:

```
0x1000: arr[0] = ?
0x1004: arr[1] = ?
0x1008: arr[2] = ?
0x100C: arr[3] = ?  <-- This address
0x1010: arr[4] = ?
```

Practice Questions

MCQ 1: What is the time complexity of accessing `arr[i]` in an array? - A) $O(n)$ - B) $O(1)$ - C) $O(\log n)$ - D) $O(n^2)$

MCQ 2: In C, when an array is passed to a function, what happens? - A) Entire array is copied - B) Array decays to a pointer - C) Compiler error - D) Array size is preserved

MCQ 3: What is the memory layout of a 2D array in C? - A) Column-major - B) Row-major - C) Random - D) Hierarchical

Short Answer: Explain why arrays have $O(1)$ access time while linked lists have $O(n)$.

Board Coding: Write a C function that takes a 2D array (3x3) and returns the sum of all elements.

Oral Explanation: What's the difference between an array and a linked list? When would you prefer one over the other?

Solutions

MCQ 1: B) $O(1)$ - Direct calculation: `address = base + i * element_size`

MCQ 2: B) Array decays to a pointer - only address of first element is passed

MCQ 3: B) Row-major - rows stored contiguously, then columns

Short Answer: Arrays store elements in contiguous memory with each element at a predictable offset ($\text{base} + i * \text{size}$). Access is one multiplication and one addition. Linked lists scatter nodes in memory; finding element at index i requires starting at head and traversing i nodes sequentially.

Board Coding:

```
int sum3x3(int matrix[3][3]) {
    int sum = 0;
    for (int i = 0; i < 3; i++) {
        for (int j = 0; j < 3; j++) {
            sum += matrix[i][j];
        }
    }
    return sum;
}
```

4.4 Records and Structures

Definition / Theoretical Background

Structures (struct) and **records** are composite data types that group together related data of different types under a single name.

Language	Term	Syntax
C/C++	struct	struct Point { int x; int y; };
C++	class	class Point { public: int x; int y; };
Pascal	record	record x, y: integer; end
Python	namedtuple/dataclass	@dataclass class Point:
Java	class	class Point { int x; int y; }

C Structures

```
// Definition
struct Point {
    int x;
    int y;
};

// Declaration
struct Point p1;           // Uninitialized
struct Point p2 = {10, 20}; // x=10, y=20

// Access members with .
p1.x = 5;
p1.y = 10;
printf("Point: (%d, %d)\n", p1.x, p1.y);
```

```
// Pointer access with ->
struct Point *ptr = &p1;
ptr->x = 15;           // Same as (*ptr).x = 15
ptr->y = 25;
```

Memory Layout of Structures

```
struct Example {
    char a;    // 1 byte
    int b;     // 4 bytes
    char c;    // 1 byte
};

// Memory might look like (with padding):
// [ a ][ padding ][ b ][ c ][ padding ]
//  1   3         4         1   3

printf("sizeof(Example) = %zu\n", sizeof(struct Example));
// Result is typically 12 bytes (not 6!) due to alignment padding
```

Structs within Structs (Nested Structures)

```
struct Rectangle {
    struct Point topLeft;    // Nested struct
    struct Point bottomRight; // Another nested struct
};

struct Rectangle rect = {
    {0, 0},    // topLeft = (0, 0)
    {100, 50}  // bottomRight = (100, 50)
};

// Access nested members:
printf("TL: (%d, %d)\n", rect.topLeft.x, rect.topLeft.y);
printf("BR: (%d, %d)\n", rect.bottomRight.x, rect.bottomRight.y);

// With pointer:
struct Rectangle *r = &rect;
printf("Width: %d\n", r->bottomRight.x - r->topLeft.x);
```

Typedef for Simpler Syntax

```
// Without typedef
struct Complex {
    double real;
    double imag;
```

```
};

struct Complex c1, c2; // Must use 'struct' keyword

// With typedef
typedef struct {
    double real;
    double imag;
} Complex;

Complex c1, c2; // Simpler, no 'struct' needed
```

Edge Cases & Traps

Trap 1: Struct Size and Padding

```
struct A {
    char a;
    char b;
    char c;
    char d;
};
// sizeof = 4 (no padding needed)

// vs

struct B {
    int x;
    char c;
};
// sizeof might be 8 (4 for int + 1 for char + 3 padding)
```

[WARNING] **Professor Trap:** Structure size is NOT the sum of member sizes due to alignment padding!

Trap 2: Comparing Structs with ==

```
struct Point p1 = {10, 20};
struct Point p2 = {10, 20};

// WRONG:
if (p1 == p2) { } // ERROR! Cannot compare structs with ==

// Must compare member by member:
if (p1.x == p2.x && p1.y == p2.y) { } // Correct
```

Trap 3: Returning Struct vs Pointer

```
struct Point createPoint(int x, int y) {
    struct Point p = {x, y};
```

```

    return p; // Returns COPY (whole struct copied)
}

// vs

struct Point *createPointPtr(int x, int y) {
    struct Point *p = malloc(sizeof(struct Point));
    p->x = x;
    p->y = y;
    return p; // Returns pointer to heap-allocated struct
}

// First version copies struct (may be slow for large structs)
// Second version avoids copy but requires manual memory management

```

Trap 4: Uninitialized Struct

```

struct Point p; // UNINITIALIZED! Contains garbage values!

struct Point p = {0}; // ALL members set to 0 (correct!)

```

Examples

```

struct Node {
    int data;
    struct Node *next; // Self-referential pointer!
};

struct Node node1 = {10, NULL};
struct Node node2 = {20, NULL};
struct Node node3 = {30, NULL};

// Link them
node1.next = &node2;
node2.next = &node3;

// Now we have: node1 -> node2 -> node3 -> NULL

// Traverse
struct Node *current = &node1;
while (current != NULL) {
    printf("%d -> ", current->data);
    current = current->next;
}

// Output: 10 -> 20 -> 30 ->

```

Example: Using Structs for Linked List Node

Practice Questions

MCQ 1: What is the size of `struct { char a; int b; }` on a 32-bit system with 4-byte int? - A) 5 bytes - B) 8 bytes - C) 4 bytes - D) Depends on compiler

MCQ 2: How do you access member `y` of a struct pointer `p`? - A) `p.y` - B) `p->y` - C) `(*p).y` - D) Both B and C

Short Answer: Why can't we compare two structs with `==` in C?

Solutions

MCQ 1: B) 8 bytes (char 1 + 3 padding + int 4)

MCQ 2: D) Both B and C - `p->y` is syntactic sugar for `(*p).y`

Short Answer: Structures contain multiple fields of potentially different sizes. The compiler doesn't know how to compare them efficiently. You must compare member by member.

Cheat-Sheet

Concept	Key Points
Array	Contiguous memory, fixed size, $O(1)$ access
List	Scattered memory, dynamic size, $O(n)$ access
Array Decay	Array becomes pointer when passed to function
Row-Major (C)	Elements stored row by row
Column-Major (Fortran)	Elements stored column by column
Struct Padding	Extra bytes added for alignment
Struct vs Class	Struct members public by default, class private

Review Checklist

- ☐ **Array Access:** Can you explain why `arr[i]` is $O(1)$?
- ☐ **Array vs List:** Can you contrast fixed vs dynamic size, contiguous vs scattered?
- ☐ **Decay:** Can you explain why `sizeof` in function gives pointer size?
- ☐ **2D Arrays:** Can you trace element address in row-major memory?
- ☐ **Struct Padding:** Do you know why `sizeof(struct)` may be larger than sum of members?
- ☐ **Traps:** Are you aware of off-by-one, bounds checking, and comparison traps? # Chapter 4 Part 4: Pointers

4.5 Pointers

Definition / Theoretical Background

A **pointer** is a variable that stores a memory address of another variable. Pointers enable: - Indirect access to data - Dynamic memory allocation - Passing large structures efficiently - Building complex data structures (linked lists, trees)

KEY CONCEPT: A pointer is a variable that holds a **memory address**, not the actual value.

Pointer Fundamentals

Memory Model (Revisited)

Variable and Pointer in Memory:

```
int x = 42;           // x is stored at address 0x1000, value = 42
int *ptr = &x;        // ptr is stored at address 0x2000, value = 0x1000 (address of x)
```

Memory Map:

Address	Content	Variable Name
0x1000	42	x
0x1004	0x1000	ptr (the value stored)
	...	

But ptr itself has an address (0x2000)!

```
// Declaration: type *name;
int *ptr1;           // Pointer to int (uninitialized - garbage address!)
float *ptr2;         // Pointer to float
char *ptr3;          // Pointer to char

// Initialize with address of another variable using &
int x = 10;
int *ptr = &x;       // ptr now holds address of x

// Multiple pointers in one declaration
int *p1, *p2, *p3;   // All are pointers
int *p4, p5;         // p4 is pointer, p5 is plain int!
```

Declaration and Initialization

```
int x = 5;
printf("%p", &x); // Prints address of x (e.g., 0x7ffd...)

// & gives the ADDRESS, * gives the VALUE at that address
```

The Address-of Operator (&)

Reference and Dereference

These are the TWO fundamental pointer operations:

Operation	Symbol	Meaning
Reference (address-of)	&	Get address of a variable
Dereference (indirection)	*	Get value at an address

```
int x = 42;
int *ptr = &x;  // &x gives the address of x
                // ptr now "points to" x

// ptr contains: 0x1000 (address of x)
// ptr's own address: 0x2000 (different!)
```

Reference: Getting a Pointer

```
int x = 42;
int *ptr = &x;

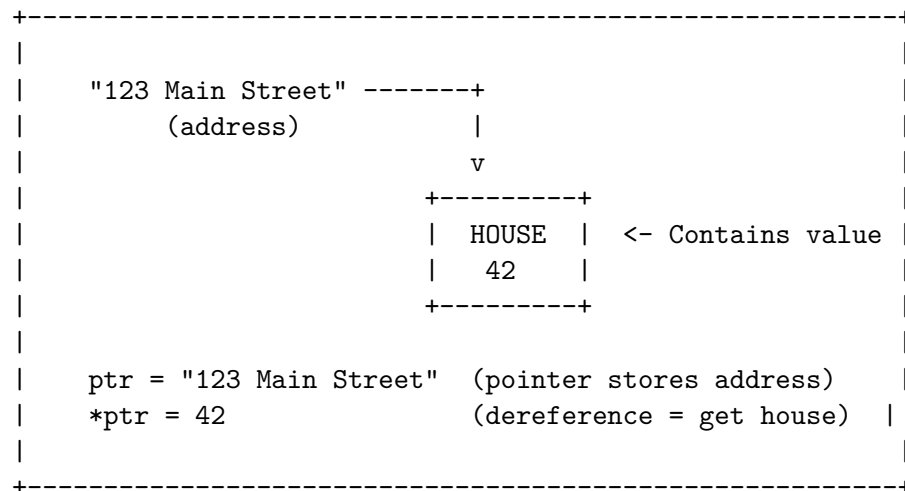
// Dereference ptr to get the value at x
printf("%d", *ptr);  // Prints 42 (value at address ptr points to)

// Modify value through pointer
*ptr = 100;          // Changes x to 100!
printf("%d", x);     // Prints 100
```

Dereference: Accessing the Value

Pointer is Like a House Address

Think of a pointer as a SIGN POST:



Pointer Arithmetic

Pointers can be incremented, decremented, and have arithmetic performed on them.

IMPORTANT: Pointer arithmetic automatically scales by the size of the pointed-to type!

```
int arr[5] = {10, 20, 30, 40, 50};
int *ptr = arr; // Points to arr[0]

// ptr + 1 moves to next int, not just address + 1!
printf("%p\n", ptr); // e.g., 0x1000
printf("%p\n", ptr + 1); // e.g., 0x1004 (moved 4 bytes for int)
printf("%p\n", ptr + 2); // e.g., 0x1008

// Similarly for char*
char str[] = "Hello";
char *cptr = str;
printf("%p\n", cptr); // 0x1000
printf("%p\n", cptr + 1); // 0x1001 (moved 1 byte for char)

// Accessing array elements via pointer arithmetic
printf("%d\n", *(ptr + 2)); // 30 (same as ptr[2])
printf("%d\n", ptr[2]); // 30 (same as *(ptr + 2))
```

```
int arr[5] = {10, 20, 30, 40, 50};
int *ptr = arr; // ptr = base address, arr[i] = *(arr + i)

// Operation      Result      Explanation
// -----
// ptr + 1        Address + 4    Next int (if int is 4 bytes)
// ptr - 1        Address - 4    Previous int
// ptr++          Move to next int  Post-increment
// ++ptr          Move to next int  Pre-increment
// ptr - arr      Integer (2)      Distance between pointers
// &arr[i+1] - &arr[i] Integer (1) Number of elements between
```

Pointer Arithmetic Table

Single Pointers: Complete Example

```
#include <stdio.h>

int main() {
    int x = 10;
    int *ptr = &x;

    // Printing all relevant values
    printf("x value:      %d\n", x); // 10
```

```

printf("x address:      %p\n", (void*)&x);    // 0x7ffd...
printf("ptr value:      %p\n", (void*)ptr);    // Same as &x
printf("ptr address:    %p\n", (void*)&ptr);    // Different address
printf("*ptr (dereference): %d\n", *ptr);      // 10

// Modify x through pointer
*ptr = 20;
printf("x after *ptr=20: %d\n", x);           // 20

// Pointer arithmetic
int arr[5] = {100, 200, 300, 400, 500};
int *aptr = arr;

printf("*aptr:         %d\n", *aptr);         // 100
printf("*(aptr+1): %d\n", *(aptr + 1));       // 200
printf("*(aptr+4): %d\n", *(aptr + 4));       // 500

// aptr[i] is same as *(aptr + i)
printf("aptr[3]:      %d\n", aptr[3]);        // 400

return 0;
}

```

Double Pointers (Pointer to Pointer)

A double pointer stores the address of another pointer.

Memory Layout for Double Pointer:

```

int x = 42;
int *ptr1 = &x;    // ptr1 points to x
int **ptr2 = &ptr1; // ptr2 points to ptr1

```

Diagram:

```

+-----+
|
|  x at 0x1000:  42
|
|  ptr1 at 0x2000: 0x1000 (holds address of x)
|
|  ptr2 at 0x3000: 0x2000 (holds address of ptr1)
|
|  To get x through ptr2: **ptr2 = 42
|
+-----+

```

Why Use Double Pointers? Reason 1: Modify the pointer itself in a function

```

// WITHOUT double pointer - can't modify original pointer!
void wrong(int *ptr) {
    ptr = malloc(sizeof(int)); // Only modifies local copy of ptr
    *ptr = 42;                 // This modifies the allocated memory
}                             // But original ptr is unchanged!

// WITH double pointer - can modify original pointer!
void correct(int **ptr) {
    *ptr = malloc(sizeof(int)); // Modifies caller's pointer
    **ptr = 42;                 // Sets value at allocated memory
}

int main() {
    int *ptr = NULL;
    correct(&ptr); // Pass address of pointer
    printf("%d", *ptr); // 42
    free(ptr);
}

```

Reason 2: Building dynamic 2D arrays

```

// Allocate 2D array dynamically
int **createMatrix(int rows, int cols) {
    int **matrix = malloc(rows * sizeof(int*));
    for (int i = 0; i < rows; i++) {
        matrix[i] = malloc(cols * sizeof(int));
    }
    return matrix;
}

```

Reason 3: Pointers to pointers in linked list / tree roots

```

// Insert at beginning of linked list
void insert(struct Node **head, int data) {
    struct Node *newNode = malloc(sizeof(struct Node));
    newNode->data = data;
    newNode->next = *head;
    *head = newNode; // Modify the head pointer itself
}

```

Pointer and Array Relationship

KEY RELATIONSHIP: Array name is a constant pointer to the first element!

```

int arr[5] = {10, 20, 30, 40, 50};
int *ptr = arr; // SAME THING: ptr = &arr[0]

// All these are equivalent:
arr[0]    == *ptr    == *(ptr + 0)

```

```

arr[1]    == *(ptr + 1) == *(arr + 1)
*(arr + i) == ptr[i]    == *(ptr + i)

// But DIFFERENCE:
sizeof(arr)    // = 5 * sizeof(int) = 20 (in bytes) - THE ACTUAL ARRAY
sizeof(ptr)    // = sizeof(int*) = 8 or 4 (POINTER SIZE)

```

NULL Pointers

NULL is a special pointer value that points to nothing (address 0).

```

int *ptr = NULL; // Explicitly NULL pointer

// Always check for NULL before dereferencing!
if (ptr != NULL) {
    printf("%d", *ptr); // Safe
}

// Dereferencing NULL is UNDEFINED BEHAVIOR (usually crashes)
int *bad = NULL;
printf("%d", *bad); // CRASH! Accessing address 0

```

When to use NULL:

```

// Initialize pointers to NULL when not yet pointing to valid memory
int *ptr = NULL; // Don't leave pointers uninitialized!

// After freeing memory, set to NULL to avoid dangling pointer
free(ptr);
ptr = NULL; // Now ptr is safe to check

// Return NULL to indicate error
int *findElement(...) {
    if (not_found) return NULL;
    // ...
}

```

Dangling Pointers

A **dangling pointer** points to memory that has been freed/deallocated.

```

int *ptr = malloc(sizeof(int));
*ptr = 42;

free(ptr); // Memory freed

// ptr is now a DANGLING pointer - pointing to freed memory!
printf("%d", *ptr); // UNDEFINED! Might print garbage or crash!

```

```
ptr = NULL; // Fix: set to NULL after free
```

Common dangling pointer scenarios:

```
// Scenario 1: Returning address of local variable
int *wrong() {
    int x = 10; // Local variable - stack frame destroyed on return
    return &x; // DANGLING! x no longer exists
}

// Scenario 2: Freeing memory then using pointer
int *ptr = malloc(sizeof(int));
*ptr = 100;
free(ptr);
printf("%d", *ptr); // DANGLING!

// Scenario 3: Aliasing - two pointers to same memory, one freed
int *p1 = malloc(sizeof(int));
int *p2 = p1;
free(p1);
printf("%d", *p2); // DANGLING! p2 still points to freed memory
```

Memory Leaks

A **memory leak** occurs when allocated memory is never freed.

```
// Memory leak example
void leak() {
    int *ptr = malloc(sizeof(int));
    *ptr = 42;
    // Missing free(ptr)! Memory leaked every time this function runs
}

// Leaks can accumulate and crash program
for (int i = 0; i < 1000000; i++) {
    int *leak = malloc(1000);
    // Never freed!
}
// Eventually: out of memory crash
```

How to prevent leaks:

```
// Always pair malloc with free
void safe() {
    int *ptr = malloc(sizeof(int));
    *ptr = 42;
    // ... use ptr ...
    free(ptr); // Free when done
}
```

```
// Use valgrind to detect leaks:  
valgrind --leak-check=full ./program
```

Pointer to Functions

```
// Function pointer type  
int (*funcPtr)(int, int); // Pointer to function taking (int, int) and returning int  
  
// Assign function to pointer  
int add(int a, int b) { return a + b; }  
funcPtr = add; // funcPtr now points to add  
  
// Call through pointer  
int result = funcPtr(5, 3); // Same as add(5, 3)  
  
// Use cases: callbacks, polymorphism, state machines
```

Void Pointers (Generic Pointers)

```
void *ptr; // Can point to ANY type  
  
int x = 5;  
float f = 3.14;  
  
void *vptr = &x; // Points to int  
vptr = &f; // Now points to float  
  
// Must cast before dereferencing  
int *iptr = (int*)vptr;  
printf("%d", *iptr);
```

Edge Cases & Traps

Trap 1: Uninitialized Pointers (MOST DANGEROUS!)

```
int *ptr; // Points to RANDOM address (garbage)!  
  
// This is UNDEFINED BEHAVIOR - could crash or corrupt memory!  
*ptr = 42; // Writing to unknown memory location!  
  
// FIX: Always initialize  
int *ptr = NULL; // or  
int *ptr = &existingVariable;
```

Trap 2: Dereferencing NULL

```

int *ptr = NULL;
printf("%d", *ptr);  // CRASH! NULL is address 0, not valid!

// ALWAYS check before dereference:
if (ptr != NULL) {
    printf("%d", *ptr);
}

```

Trap 3: Pointer vs Value Confusion

```

int x = 10, y = 20;
int *p1 = &x;
int *p2 = &y;

*p1 = *p2;  // Changes x to 20 (dereference both!)
p1 = p2;    // Changes p1 to point to y (pointer assignment!)

// What is the difference?
// *p1 = *p2  ->  x = y (values)
// p1 = p2    ->  p1 and p2 point to same location

```

Trap 4: Pointer Arithmetic on Different Types

```

int *iptr;
char *cptr;

printf("%p", iptr + 1);  // Adds sizeof(int) bytes
printf("%p", cptr + 1);  // Adds sizeof(char) = 1 byte

// What if we add int to char pointer by mistake?
char *wrong = (char*)((int*)iptr + 1);  // Wrong! Adds sizeof(int) not 1 byte!

```

Trap 5: The Famous i++ + ++i With Pointers (UNDEFINED!)

```

int arr[5] = {1, 2, 3, 4, 5};
int *p = arr;

int result = *p++ + ++*p;  // UNDEFINED! p modified twice!

// What might happen:
// If left evaluated first: *(p++) + *(++p) = 1 + 3 = 4 (if p moves correctly)
// If right evaluated first: *(++p) + *(p++) = 3 + 2 = 5
// OR different values!

// Safe way - separate statements:
int result = *p;
p++;
result += *p;
p++;

```

Trap 6: Comparing Pointers (Wrong Way)

```
int arr1[5] = {1, 2, 3, 4, 5};
int arr2[5] = {1, 2, 3, 4, 5};

int *p1 = arr1;
int *p2 = arr2;

if (p1 == p2) { } // NEVER equal - different arrays, different addresses!

// Pointers to SAME array can be meaningfully compared:
int *p3 = arr1;
int *p4 = arr1 + 3;
if (p3 < p4) { } // VALID - p3 is before p4 in same array
```

Trap 7: &array vs array

```
int arr[5] = {1, 2, 3, 4, 5};

printf("%p\n", arr); // Address of first element (same as &arr[0])
printf("%p\n", &arr); // Address of entire array!

// But they have different types:
printf("%zu\n", sizeof(arr)); // 20 (entire array)
printf("%zu\n", sizeof(&arr)); // 8 (pointer size!)

// And pointer arithmetic differs:
printf("%zu\n", sizeof(arr + 0)); // 8 (arr decays to pointer)
```

Trap 8: String Literal Assignment

```
char *s1 = "Hello"; // Points to READ-ONLY memory (undefined behavior to modify)
char s2[] = "Hello"; // Copy of string, MODIFIABLE

s1[0] = 'h'; // UNDEFINED! May crash!
s2[0] = 'h'; // OK! s2 is a modifiable array

// Why? "Hello" is a string literal stored in read-only memory
// s1 points to it, s2 copies it into writable stack memory
```

Trap 9: Casting Pointers Incorrectly

```
int x = 0x41424344; // 'ABCD'
char *cptr = (char*)&x;

printf("%c", cptr[0]); // 'D' (depending on endianness!)
printf("%c", cptr[1]); // 'C'
printf("%c", cptr[2]); // 'B'
printf("%c", cptr[3]); // 'A'
```

Examples

```
// Swap two variables using pointers
void swap(int *a, int *b) {
    int temp = *a;    // temp = value at a
    *a = *b;          // value at a = value at b
    *b = temp;        // value at b = temp
}

int main() {
    int x = 5, y = 10;
    printf("Before: x=%d, y=%d\n", x, y); // x=5, y=10
    swap(&x, &y); // Pass ADDRESSES
    printf("After: x=%d, y=%d\n", x, y); // x=10, y=5
}
```

Example 1: Pointer Swap (Important Exam Pattern!)

```
int sumArray(int *arr, int n) {
    int sum = 0;
    int *ptr = arr;
    int *end = arr + n;

    while (ptr < end) {
        sum += *ptr; // Add value at ptr
        ptr++;      // Move to next element
    }
    return sum;
}

int main() {
    int arr[] = {1, 2, 3, 4, 5};
    printf("%d\n", sumArray(arr, 5)); // 15
}
```

Example 2: Array Sum Using Pointer Arithmetic

```
#include <stdio.h>
#include <stdlib.h>

void allocateInt(int **ptr, int value) {
    *ptr = malloc(sizeof(int));
    **ptr = value; // Set the allocated memory to value
}

int main() {
```

```

int *ptr = NULL;
allocateInt(&ptr, 42); // Pass address of ptr
printf("%d\n", *ptr); // 42
free(ptr);
}

```

Example 3: Double Pointer to Modify Pointer Value

Example 4: Pointer Walk Through Memory (White-Board Style)

Question: What does this print?

```

int arr[] = {10, 20, 30, 40, 50};
int *p = arr;

printf("%d ", *(p + 2)); // ?
printf("%d ", *p + 2);   // ?
printf("%d ", *(++p));   // ?
printf("%d ", *p++);     // ?

```

Step-by-step:

```

*(p + 2) = arr[2] = 30
*p + 2 = 10 + 2 = 12 (p unchanged)
*(++p) = *(arr + 1) = 20 (p now points to arr[1])
*p++ = arr[1] = 20 (prints, THEN increments p to arr[2])

```

Final output: 30 12 20 20
p now points to arr[2]

```

// Find ALL bugs in this code:

int *createArray(int size) {
    int arr[size]; // VLA - local, on stack!
    for (int i = 0; i < size; i++) {
        arr[i] = i;
    }
    return arr; // BUG: returning address of local (freed) memory!
}

// FIXED version:
int *createArrayFixed(int size) {
    int *arr = malloc(size * sizeof(int)); // Heap allocation
    for (int i = 0; i < size; i++) {
        arr[i] = i;
    }
    return arr; // OK: heap memory persists after return
}

```

```

}

// Caller:
int *myArr = createArrayFixed(10);
// Use myArr...
free(myArr); // Don't forget to free!

```

Example 5: Finding Errors in Pointer Code

Practice Questions

MCQ 1: What does `int *ptr;` declare? - A) An integer variable named ptr - B) A pointer to an integer - C) An integer that points somewhere - D) A reference to ptr

MCQ 2: What is the result of `*&x` (assuming x is an int variable)? - A) The address of x - B) The value of x - C) A pointer to x - D) Error

MCQ 3: What is a dangling pointer? - A) A pointer that hasn't been initialized - B) A pointer to freed/deallocated memory - C) A pointer to NULL - D) A pointer that leaks memory

Short Answer: Explain why `*ptr++` is different from `(*ptr)++`.

Board Coding: Write a function that takes a double pointer to int and allocates a 2D array of ints.

Oral Explanation: What is the difference between passing a pointer to a function vs passing a value? Why would you use pointers?

Solutions

MCQ 1: B) A pointer to an integer

MCQ 2: B) The value of x - `*&x` means “value at address of x” = x

MCQ 3: B) Pointer to freed/deallocated memory

Short Answer: `*ptr++` increments the pointer (moves to next element), returning the OLD value. `(*ptr)++` increments the value at the pointer, returning the OLD value. The difference: one changes WHERE ptr points, the other changes WHAT ptr points to.

Board Coding:

```

void allocate2D(int ***arr, int rows, int cols) {
    *arr = malloc(rows * sizeof(int*));
    for (int i = 0; i < rows; i++) {
        (*arr)[i] = malloc(cols * sizeof(int));
    }
}

// Usage:
int **matrix;
allocate2D(&matrix, 3, 3);

```

```
matrix[0][0] = 1;
free(matrix); // Don't forget!
```

Cheat-Sheet

Concept	Key Points
Pointer	Variable holding a memory address
& (reference)	Get address of variable
* (dereference)	Get value at address
NULL	Pointer pointing to nothing (address 0)
Dangling	Points to freed memory
Memory leak	Allocated memory never freed
Double pointer	Pointer to pointer, used to modify pointers
Pointer arithmetic	Automatically scales by sizeof(type)
Array decay	Array name is constant pointer to first element

Review Checklist

- ☐ **Reference/Dereference:** Can you explain & and * operations?
- ☐ **Pointer arithmetic:** Do you understand scaling by sizeof(type)?
- ☐ **Double pointers:** Can you explain why we need them?
- ☐ **NULL:** Do you always check before dereferencing?
- ☐ **Dangling pointers:** Can you identify and fix them?
- ☐ **Memory leaks:** Do you always pair malloc with free?
- ☐ **Common traps:** Uninitialized pointers, pointer vs value confusion, string literals # Chapter 5: Memory Model and Subprograms

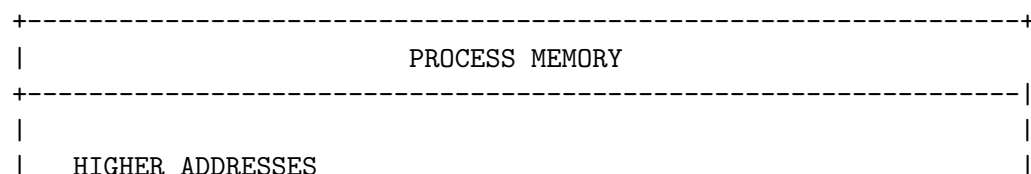
5.1 Stack, Heap, Static (Lifetime)

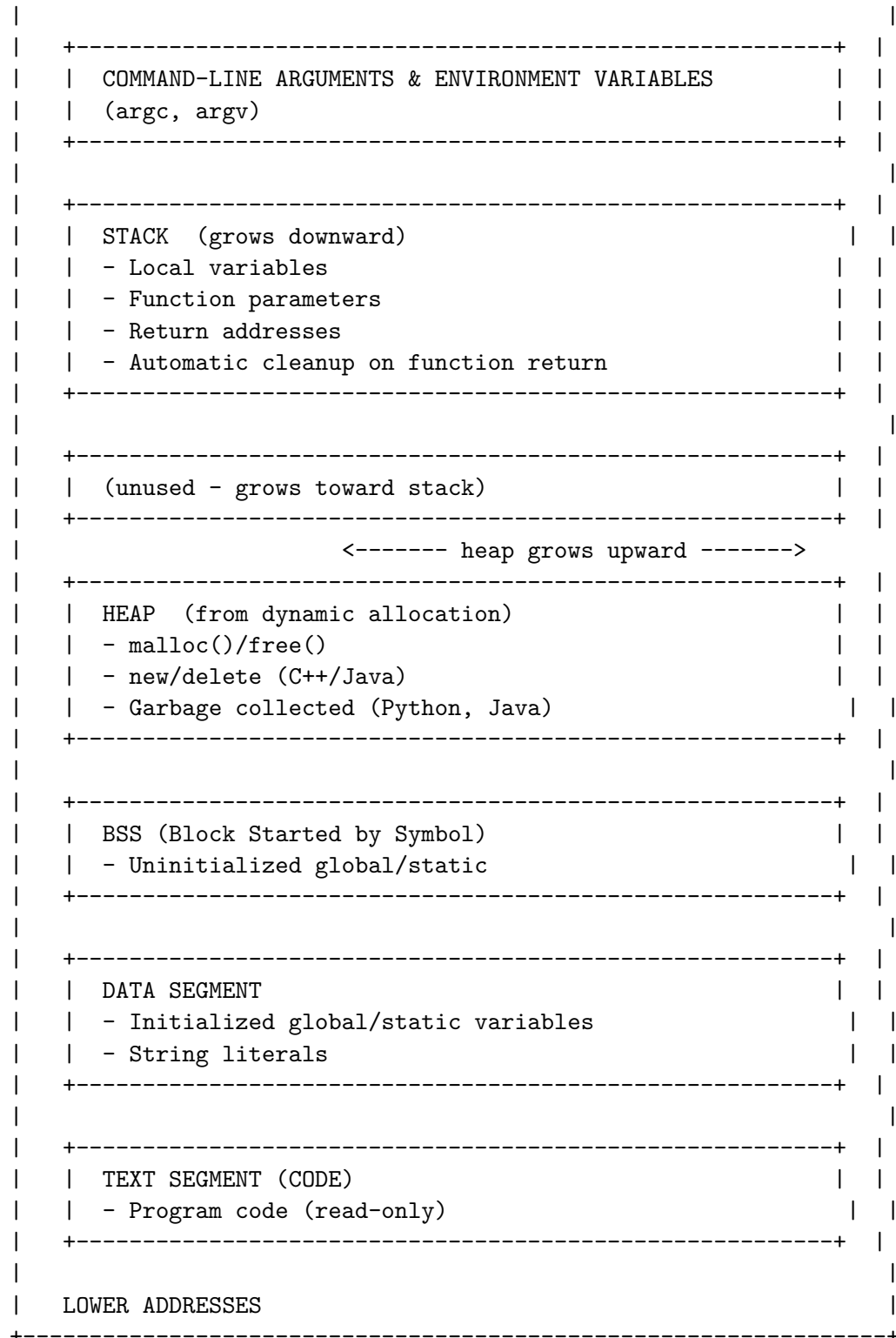
Definition / Theoretical Background

Memory Lifetime refers to how long a variable exists in memory before being destroyed.

Storage Class	Lifetime	Where	Example
Automatic	Until block ends	Stack	Local variables
Static	Whole program	Data segment	Global, static
Dynamic	Until freed	Heap	malloc/free

Memory Layout Diagram





The Stack

The stack is a region of memory that stores: - Function parameters - Local variables - Return addresses - Saved registers

Key properties: - **Grows downward** (from high to low addresses) - **Automatic cleanup** when function returns - **Fast allocation** (just move stack pointer) - **Limited size** (typically 1-8 MB)

```
void foo(int x) {           // x is on stack
    int y = 10;             // y is on stack
}

int main() {
    foo(5);                 // foo's stack frame created
    return 0;               // foo's stack frame destroyed
}
```

Stack Frame (Activation Record)

When foo() is called:

Stack (top at bottom):

```
+-----+
| Return address (main) | <- Where to return after foo
+-----+
| Parameter: x = 5      | <- Function parameter
+-----+
| Local var: y = 10     | <- Local variable
+-----+
| Saved registers       |
+-----+
```

Concrete Memory Address Examples (IMPORTANT FOR EXAMS!)

Example: Variable Addresses in Memory

```
int x = 42;           // Stored at address 0x1000 (4 bytes)
char c = 'A';         // Stored at address 0x1004 (1 byte)
float f = 3.14;       // Stored at address 0x1008 (4 bytes)
double d = 2.71;      // Stored at address 0x100C (8 bytes)

printf("x at %p = %d\n", (void*)&x, x); // x at 0x1000 = 42
printf("c at %p = %c\n", (void*)&c, c); // c at 0x1004 = A
printf("f at %p = %f\n", (void*)&f, f); // f at 0x1008 = 3.14
printf("d at %p = %lf\n", (void*)&d, d); // d at 0x100C = 2.71

// Memory layout (assuming little-endian, 32-bit int):
// Address: 0x1000 0x1001 0x1002 0x1003 0x1004
// Content: [ 42 ] [ 0 ] [ 0 ] [ 0 ] [ 65 ]
//           x (int, little-endian: 2A 00 00 00)
//           c (char: 41 = 'A')
```

Example: Array Memory Layout with Addresses

```

int arr[5] = {10, 20, 30, 40, 50};
// Assuming arr starts at 0x2000, int is 4 bytes:

printf("arr[0] at %p = %d\n", (void*)&arr[0], arr[0]); // 0x2000 = 10
printf("arr[1] at %p = %d\n", (void*)&arr[1], arr[1]); // 0x2004 = 20
printf("arr[2] at %p = %d\n", (void*)&arr[2], arr[2]); // 0x2008 = 30
printf("arr[3] at %p = %d\n", (void*)&arr[3], arr[3]); // 0x200C = 40
printf("arr[4] at %p = %d\n", (void*)&arr[4], arr[4]); // 0x2010 = 50

// Visual memory:
Address: 0x2000 0x2004 0x2008 0x200C 0x2010
Value:  | 10 | 20 | 30 | 40 | 50 |
Index:  | [0] | [1] | [2] | [3] | [4] |

// Formula: &arr[i] = base_address + (i * sizeof(int))
// &arr[3] = 0x2000 + (3 * 4) = 0x2000 + 12 = 0x200C

```

Example: Pointer Values as Addresses

```

int x = 42;
int *ptr = &x;

printf("Address of x (&x): %p\n", (void*)&x); // e.g., 0x1000
printf("Value of ptr: %p\n", (void*)ptr); // 0x1000 (same!)
printf("Address of ptr (&ptr):%p\n", (void*)&ptr); // e.g., 0x2000 (different!)
printf("Value at *ptr: %d\n", *ptr); // 42

// Memory visualization:
0x1000: [ 42 ] <- x (value)
0x1004: [ ?? ] <- (next variable)

0x2000: [ 0x1000 ] <- ptr (stores ADDRESS of x as its value!)
0x2008: [ ?? ] <- (next variable)

// ptr at 0x2000 stores 0x1000 (the address of x)
// *ptr dereferences: goes to address stored in ptr, gets value 42

```

Example: Heap Allocation with Addresses

```

int *heap1 = (int*)malloc(sizeof(int)); // Allocate 4 bytes on heap
int *heap2 = (int*)malloc(sizeof(int)); // Allocate 4 bytes on heap

*heap1 = 100;
*heap2 = 200;

printf("heap1 points to address: %p\n", (void*)heap1); // e.g., 0x5000

```

```

printf("heap2 points to address: %p\n", (void*)heap2); // e.g., 0x5008
printf("*heap1 = %d, *heap2 = %d\n", *heap1, *heap2); // 100, 200

// Heap grows upward (low to high):
// 0x5000: [ 100 ] <- heap1 points here
// 0x5004: [ ?? ] <- (next available, if any)
// 0x5008: [ 200 ] <- heap2 points here

free(heap1); // 0x5000 returned to free memory pool
free(heap2); // 0x5008 returned to free memory pool

```

Example: Execution Stack with Real Addresses

```

// Assume:
// - main at 0x1000 (code)
// - foo at 0x2000 (code)
// - Stack starts at 0xFF00 and grows DOWN

void foo(int p) { // p = 5
    int local = 10; // local variable
    // When foo is called, stack looks like:
}

int main() {
    int num = 5;
    foo(num); // Call foo with 5
}

/*
Execution Stack when inside foo():

HIGH ADDRESS: 0xFF00
    |
    | <- Stack pointer (SP)
    |
    +-----+
    | Return Addr: 0x1040 | <- Where to return in main
    +-----+
    | Saved FP: 0xFEFO    | <- Previous frame pointer
    +-----+
    | Parameter: p = 5    | <- Address 0xFEC8 (4 bytes)
    +-----+
    | Local var: local = 10 | <- Address 0xFEC4 (4 bytes)
    +-----+
    | (next local if any) |
    |
LOW ADDRESS: 0xFE00
*/

```

```
// IMPORTANT: Stack grows DOWN (to lower addresses)
// When foo() calls another function, SP decreases (subtracts)
```

Example: Pointer Arithmetic with Addresses (CRITICAL!)

```
int arr[5] = {10, 20, 30, 40, 50};
int *p = arr;    // p = &arr[0] = 0x3000

printf("p      = %p\n", (void*)p);           // 0x3000
printf("p + 1  = %p\n", (void*)(p + 1));     // 0x3004 (added 4 bytes for int)
printf("p + 2  = %p\n", (void*)(p + 2));     // 0x3008 (added 8 bytes total)
printf("*p     = %d\n", *p);                  // 10 (value at 0x3000)
printf("*(p+1) = %d\n", *(p + 1));           // 20 (value at 0x3004)
printf("*(p+4) = %d\n", *(p + 4));           // 50 (value at 0x3010)

/*
Memory layout:
0x3000: [10] <- *p
0x3004: [20] <- *(p+1)
0x3008: [30] <- *(p+2)
0x300C: [40] <- *(p+3)
0x3010: [50] <- *(p+4)

Key insight: p + 1 adds sizeof(int) = 4 bytes, NOT 1 byte!
Formula: *(p + i) is at address (p + i * sizeof(type))
*/

// What if it was char*?
char str[] = "ABC";
char *cp = str;    // cp = 0x4000

printf("cp      = %p\n", (void*)cp);         // 0x4000
printf("cp + 1  = %p\n", (void*)(cp + 1));   // 0x4001 (added 1 byte!)
printf("cp + 2  = %p\n", (void*)(cp + 2));   // 0x4002 (added 2 bytes!)

/*
char is 1 byte, so char* + 1 adds only 1 byte:
0x4000: ['A']
0x4001: ['B']
0x4002: ['C']
0x4003: ['\0']

This is why pointer arithmetic is TYPE-DEPENDENT!
*/
```

Example: Double Pointer Memory Layout

```

int x = 42;
int *ptr1 = &x;           // ptr1 = 0x1000 (address of x)
int **ptr2 = &ptr1;       // ptr2 = 0x2000 (address of ptr1)

printf("x value:         %d\n", x);           // 42
printf("*ptr1 value:    %d\n", *ptr1);        // 42 (value at address ptr1)
printf("**ptr2 value:   %d\n", **ptr2);        // 42 (value at address pointed to by ptr2)

printf("&x = %p\n", (void*)&x);               // 0x1000
printf("ptr1 = %p\n", (void*)ptr1);           // 0x1000 (same as &x)
printf("&ptr1 = %p\n", (void*)&ptr1);          // 0x2000
printf("ptr2 = %p\n", (void*)ptr2);           // 0x2000 (same as &ptr1)
printf("&ptr2 = %p\n", (void*)&ptr2);          // 0x3000

/*
Memory Layout:
0x1000: [ 42 ] <- x (int)
0x1004: [ ?? ] <- next int (if any)

0x2000: [ 0x1000 ] <- ptr1 (stores address of x)
0x2008: [ ?? ] <- next pointer (if any)

0x3000: [ 0x2000 ] <- ptr2 (stores address of ptr1)
0x3008: [ ?? ] <- next pointer (if any)

To get x through ptr2: **ptr2
- ptr2 at 0x3000 = 0x2000
- *ptr2 = value at 0x2000 = 0x1000 (which is ptr1)
- **ptr2 = value at address 0x1000 = 42
*/

```

Example: Stack vs Heap - Key Difference

```

void example() {
    int stackVar = 10;           // On stack
    int *heapPtr = malloc(sizeof(int)); // On heap
    *heapPtr = 10;

    printf("stackVar at %p\n", (void*)&stackVar); // e.g., 0xFF00 (HIGH address)
    printf("heapPtr at %p\n", (void*)heapPtr);   // e.g., 0x5000 (LOW address)

    /*
    Stack grows DOWN to LOWER addresses:
    0xFF00: [stackVar = 10]

    Heap grows UP to HIGHER addresses:
    0x5000: [10] <- heapPtr points here
    */

```

```

    Stack: 0xFF00, 0xFE00, 0xFD00... (high to low)
    Heap:  0x5000, 0x5010, 0x5020... (low to high)
    */

    free(heapPtr); // MUST free heap memory!
} // stackVar automatically destroyed here

### The Heap

**The heap** is a region of memory for dynamic allocation:
- **Grows upward** (from low to high addresses)
- **Manual management** (malloc/free in C)
- **Larger size** (limited by system RAM + swap)
- **Slower allocation** (search for free block)

```c
// C dynamic allocation
int *ptr = (int*)malloc(sizeof(int)); // Allocate on heap
*ptr = 42; // Use the memory
free(ptr); // Release memory
ptr = NULL; // Avoid dangling pointer

```

## Static Storage

Static variables exist for the entire program lifetime: - **Stored in data segment** (initialized) or **BSS** (uninitialized) - **Global scope** but limited to declaring file (with static keyword) - **Retained** between function calls

```

int globalVar = 10; // Static storage (data segment)
static int fileStatic; // Static storage, file scope only

void counter() {
 static int count = 0; // Static local - retains value between calls!
 count++;
 printf("Count: %d\n", count);
}

int main() {
 counter(); // Count: 1
 counter(); // Count: 2
 counter(); // Count: 3
}

```

## Comparison: Stack vs Heap vs Static

Property	Stack	Heap	Static
<b>Lifetime</b>	Until function returns	Until free() called	Whole program

Property	Stack	Heap	Static
<b>Management</b>	Automatic	Manual (malloc/free)	Automatic
<b>Speed</b>	Fast	Slower	Fast
<b>Size</b>	Limited (1-8 MB)	Large (RAM dependent)	Fixed at compile
<b>Access</b>	Local only	Via pointer	Global access
<b>Memory</b>	Contiguous	Scattered blocks	Fixed location

## Edge Cases & Traps

### Trap 1: Stack Overflow

```
// Recursive without base case - STACK OVERFLOW!
int factorial(int n) {
 return n * factorial(n - 1); // Never stops!
 // Each call adds to stack until it overflows!
}

// Also: Large local arrays go on stack
void danger() {
 int huge[1000000]; // 4MB on stack - might overflow!
 // Stack typically only 1-8MB total!
}
```

[WARNING] **Professor Trap:** Stack overflow causes crash! Don't allocate large arrays on stack!

### Trap 2: Returning Address of Local Variable

```
int *wrong() {
 int x = 10; // x is on stack
 return &x; // BUG! x is destroyed when function returns!
} // Returning address of freed memory!

// CORRECT - use heap or static:
int *correct() {
 int *x = malloc(sizeof(int));
 *x = 10;
 return x; // OK - heap memory persists after return
}

// OR use static:
int *correctStatic() {
 static int x = 10; // Static - persists!
 return &x; // OK - x won't be destroyed
}
```

[WARNING] **Professor Trap:** Never return address of local (stack) variable!

### Trap 3: Memory Leak

```

void leak() {
 int *ptr = malloc(sizeof(int));
 *ptr = 42;
 // BUG: no free(ptr)! Memory leaked every call!
}

// FIXED:
void noLeak() {
 int *ptr = malloc(sizeof(int));
 *ptr = 42;
 free(ptr); // Free before returning!
}

```

[WARNING] **Professor Trap:** Every malloc() should have a free()!

#### Trap 4: Double Free

```

int *ptr = malloc(sizeof(int));
*ptr = 42;
free(ptr);
free(ptr); // UNDEFINED! Already freed!

```

[WARNING] **Professor Trap:** Don't free the same memory twice!

#### Trap 5: Using After Free (Dangling Pointer)

```

int *ptr = malloc(sizeof(int));
*ptr = 42;
free(ptr);
printf("%d", *ptr); // UNDEFINED! ptr points to freed memory!

```

[WARNING] **Professor Trap:** After free(), pointer becomes dangling. Set to NULL!

#### Examples

```

#include <stdio.h>
#include <stdlib.h>

int global = 100; // Static - lifetime whole program

void lifetimeDemo() {
 int local = 10; // Auto (stack) - destroyed on return
 static int staticLocal = 10; // Static - persists!
 int *heapVar = malloc(sizeof(int)); // Heap - manual lifetime

 *heapVar = 20;
 printf("global=%d, local=%d, static=%d, *heap=%d\n",
 global, local, staticLocal, *heapVar);
}

```

```

 local++;
 staticLocal++;
 (*heapVar)++;

 free(heapVar); // Don't forget!
}

int main() {
 printf("Call 1:\n"); lifetimeDemo(); // g=100, l=10, s=10, h=20
 printf("Call 2:\n"); lifetimeDemo(); // g=100, l=10 (NEW), s=11 (KEPT!), h=20 (NEW)
 // Note: staticLocal keeps its value between calls!
}

```

### Example 1: Lifetime Demonstration

```

#include <stdlib.h>

int main() {
 // malloc - allocate raw memory
 int *arr = (int*)malloc(5 * sizeof(int));
 for (int i = 0; i < 5; i++) arr[i] = i;
 free(arr); // Single free for whole block

 // calloc - allocate AND zero-initialize
 int *arr2 = (int*)calloc(5, sizeof(int)); // All zeros
 // arr2[0] through arr2[4] are all 0
 free(arr2);

 // realloc - resize existing allocation
 int *arr3 = (int*)malloc(3 * sizeof(int));
 arr3 = realloc(arr3, 10 * sizeof(int)); // Now size 10!
 // Note: realloc may move data to new location
 free(arr3);

 return 0;
}

```

### Example 2: malloc/calloc/free

## 5.2 Storing Variables in Memory and Lifetime

### Variable Lifetime Summary

Storage Class	Keyword	Default Value	Lifetime	Scope
<b>Automatic</b>	(none)	Garbage	Block	Block

Storage Class	Keyword	Default Value	Lifetime	Scope
<b>Register</b>	register	Garbage	Block	Block
<b>Static</b>	static	Zero	Program	Block
<b>External</b>	extern	Zero	Program	Global
<b>External Static</b>	static	Zero	Program	File

## Storage Classes in C

```
void func() {
 int x; // Auto storage - uninitialized (garbage)
 auto int y = 5; // Explicit auto (rarely used keyword)
}
// x and y destroyed when func returns
```

## Automatic (Local Variables)

```
void counter() {
 static int count = 0; // Initialized once, retains value!
 count++;
 printf("Called %d times\n", count);
}

int main() {
 counter(); // Called 1 times
 counter(); // Called 2 times
 counter(); // Called 3 times
}
```

## Static Local Variables

```
int globalVar = 10; // Accessible everywhere in program

void func1() {
 globalVar = 20; // Can modify
}

void func2() {
 printf("%d", globalVar); // Can read
}
```

## Global Variables

```
static int fileOnlyVar = 10; // Only visible in this file!
```

```
// Even if other files have global var, this one is private to this file
```

## Static at File Scope

### extern Keyword

```
// file1.c
int sharedVar = 42;

// file2.c
extern int sharedVar; // Declare intent to use sharedVar from file1.c
printf("%d", sharedVar); // Prints 42
```

---

## 5.3 Parameter Passing, Execution Stack, Program Structure, Scope, and Visibility

### PASS BY VALUE vs PASS BY REFERENCE

THIS IS ONE OF THE MOST CONFUSING TOPICS - LET ME MAKE IT CRYSTAL CLEAR!

**What is PASS BY VALUE?** Pass by value: A COPY of the argument is passed to the function.

- Changes inside function affect the COPY, not the original
- Original argument is UNCHANGED

```
void increment(int x) { // x is a COPY of the argument
 x = x + 1;
 printf("Inside: x = %d\n", x); // x = 6
}

int main() {
 int num = 5;
 increment(num); // num is COPIED to x
 printf("Outside: num = %d\n", num); // num is still 5!
 // num was NEVER modified!
}
```

**Why?** Because x is just a copy - like photocopying a document and changing the copy.

BEFORE increment():

```
+-----+
| main: num = 5 (at address 0x1000) |
| |
| increment: x = ? (at address 0x2000) |
+-----+
```

CALL increment(num):

```
+-----+
```

```
| main: num = 5 (unchanged) |
| increment: x = 5 (COPY of num!) |
+-----+

```

AFTER x++ inside increment():

```
+-----+
| main: num = 5 (UNCHANGED) |
| increment: x = 6 (only x changed) |
+-----+

```

**What is PASS BY REFERENCE?** Pass by reference: The ADDRESS of the argument is passed (C uses pointers).

- Changes inside function affect the ORIGINAL
- You pass the memory address, function dereferences to modify

```
void incrementPtr(int *x) { // x is a POINTER to the original
 *x = *x + 1; // Modify the ORIGINAL through pointer
}

int main() {
 int num = 5;
 incrementPtr(&num); // Pass ADDRESS of num
 printf("num = %d\n", num); // num is now 6!
 // num WAS MODIFIED!
}
```

**Think of it as:** Giving someone your house key so they can go change your furniture.

BEFORE incrementPtr():

```
+-----+
| main: num = 5 (at address 0x1000) |
| |
| | (address) |
| v |
| incrementPtr: x = 0x1000 (pointer) |
+-----+

```

AFTER \*x++ inside incrementPtr():

```
+-----+
| main: num = 6 (CHANGED!) |
| incrementPtr: x = 0x1000 (still same) |
+-----+

```

## Complete Comparison Table

Aspect	Pass by Value	Pass by Reference
<b>What passed</b>	Copy of value	Address of variable
<b>In function parameter</b>	int x	int *x or int &x

Aspect	Pass by Value	Pass by Reference
Call syntax	func(num)	func(&num)
Original modified?	NO	YES
Memory overhead	Copy size	Just pointer size
Speed	Slower for large data	Faster (just address)

### Concrete Pass-by-Value Example with Addresses

```
void square(int x) {
 x = x * x; // x is a COPY, modifying copy only
}

int main() {
 int num = 5;
 printf("Before: num = %d at address %p\n", num, (void*)&num);
 square(num);
 printf("After: num = %d at address %p\n", num, (void*)&num);
 // num is STILL 5, address unchanged!
}

/*
Memory during square():
0x1000: [num = 5] <- main's num (UNCHANGED!)
0x2000: [x = 5] <- square's COPY

After x = x * x:
0x1000: [num = 5] <- STILL 5!
0x2000: [x = 25] <- only square's copy changed
*/
```

### Concrete Pass-by-Reference Example with Addresses

```
void squarePtr(int *x) {
 printf("x points to address %p\n", (void*)x); // x points to num's address!
 printf("*x before = %d\n", *x);
 *x = (*x) * (*x); // Modify value AT THAT ADDRESS
}

int main() {
 int num = 5;
 printf("num address = %p, value = %d\n", (void*)&num, num);
 squarePtr(&num); // Pass ADDRESS
 printf("num address = %p, value = %d\n", (void*)&num, num);
 // num is NOW 25!
}

/*
Memory during squarePtr():
```

```

0x1000: [num = 5] <- main's num
 ^
 | (x holds this address: 0x1000)
 v
0x2000: [x = 0x1000] <- squarePtr's parameter (pointer)

After *x = *x * *x:
0x1000: [num = 25] <- CHANGED! (via pointer dereference)
0x2000: [x = 0x1000] <- x still points to num's address
*/

```

## Common Confusion Clarified

**C ALWAYS passes by value!** Even pointers:

```

void wrong(int *ptr) {
 ptr = NULL; // Only modifies LOCAL copy of ptr!
}

int main() {
 int x = 5;
 int *p = &x;
 wrong(p);
 printf("%p", p); // NOT NULL! p unchanged!
}
// ptr = NULL only changed the COPY of the pointer!

// To actually modify pointer:
void correct(int **ptr) {
 *ptr = NULL; // Modifies caller's pointer!
 // Think: *ptr is the original pointer, setting *ptr = NULL modifies it
}

correct(&p); // Now p IS NULL!

// Let's trace with REAL addresses:
void allocInt(int **p) {
 printf("Inside: p = %p\n", (void*)p); // Address of caller's p
 printf("Inside: *p = %p\n", (void*)*p); // Value of caller's p (old address)
 p = (int)malloc(sizeof(int));
 printf("Inside: *p after alloc = %p\n", (void*)*p); // New address
 **p = 999;
}

int main() {
 int *ptr = NULL;
 printf("ptr before: %p\n", (void*)ptr); // (nil) or 0x0
 allocInt(&ptr);
}

```

```

 printf("ptr after: %p, *ptr = %d\n", (void*)ptr, *ptr); // 0x5000, 999
 free(ptr);
}
/*
Memory during allocInt():
0x1000: [ptr = NULL] <- main's pointer
 ^
 | (p in allocInt holds address 0x1000)
 v
0x2000: [p = 0x1000] <- allocInt's parameter (double pointer!)

After *p = malloc():
0x1000: [ptr = 0x5000] <- CHANGED! (original pointer now points to heap)
0x2000: [p = 0x1000] <- still holds address of ptr

0x5000: [999] <- allocated heap memory
*/

```

### ### Why Pass by Reference?

1. **\*\*Modify multiple values\*\***: Can't return multiple values with return
2. **\*\*Efficiency\*\***: Don't copy large structures, just pass address
3. **\*\*Share data\*\***: Two functions can modify same variable

```

```c
// Problem: Return only one value
// Want to return BOTH quotient AND remainder
void divide(int a, int b, int *quotient, int *remainder) {
    *quotient = a / b;
    *remainder = a % b;
}

int main() {
    int q, r;
    divide(10, 3, &q, &r);
    printf("10 / 3 = %d remainder %d\n", q, r);
}

```

C++ Reference Parameters (Different from Pointers!)

```

void incrementRef(int &x) { // Reference parameter
    x = x + 1; // No dereference needed!
}

int main() {
    int num = 5;
    incrementRef(num); // Pass directly, no & needed
}

```

```
    printf("%d", num); // 6!
}
```

C++ references are “aliases” - same variable, different name:

```
int a = 5;
int &b = a; // b is ANOTHER NAME for a
b = 10;    // a is also 10!
```

Scope and Visibility

Scope: Where a variable name can be referenced **Visibility:** Portions of scope where the variable is actually accessible

Types of Scope

```
int main() {
    int x = 10; // x visible from here to end of block
    {
        int y = 20; // y only visible in this inner block
        printf("%d %d", x, y); // Both OK
    }
    printf("%d", x); // y not visible here!
    // printf("%d", y); // ERROR! y out of scope
}
```

Block Scope (Local Scope)

```
void func() {
    // Labels have function scope
    goto label; // Can jump anywhere in func
label:
    printf("jumped");
}
```

Function Scope

```
int globalVar; // Visible from here to end of file

void func1() {
    globalVar = 10; // Can access global
}

void func2() {
    printf("%d", globalVar); // Can access global
}
```

File Scope (Global)

Scope Resolution Operators

```
int x = 10; // Global x

int main() {
    int x = 20; // Local x shadows global
    printf("%d", x); // Prints 20 (local)
    printf("%d", ::x); // Prints 10 (global, :: is scope resolution)
}
```

Name Hiding

```
int x = 10; // Global

void func() {
    int x = 20; // Hides global x within this function
    {
        int x = 30; // Hides the function's x
        printf("%d", x); // 30 (innermost)
    }
    printf("%d", x); // 20 (function's x)
}
printf("%d", x); // 10 (global)
```

Edge Cases & Traps

Trap 1: Shadowing Variables

```
int flag = 1;
if (flag) {
    int flag = 0; // BUG! Creates NEW variable, shadows outer flag!
    printf("%d", flag); // Prints 0 (inner), outer flag still 1
}
```

[WARNING] **Professor Trap:** Inner block variable with same name hides outer one!

Trap 2: Using Variable Before Defined

```
int main() {
    printf("%d", x); // ERROR! x used before declaration!
    int x = 10;
}
```

Trap 3: Static Local Variable Initialization

```
void func() {
    static int x = 5;
    x++;
}
```

```

    printf("%d", x); // 6, 7, 8, 9... each call
}

// Initialized only ONCE, not every call
// But static initialized at program start (before main)!

```

Trap 4: Global Static vs Regular Global

```

// file1.c
static int fileOnly = 10; // Only visible in file1.c
int global = 20;          // Visible everywhere

// file2.c
extern int fileOnly; // ERROR! fileOnly not visible here
extern int global;   // OK - can access

```

Trap 5: Parameter Shadowing

```

void func(int x) {
    int x = 10; // ERROR! Parameter x already declared in this scope!
}

```

Examples

```

void swap(int *a, int *b) {
    int temp = *a;
    *a = *b;
    *b = temp;
}

int main() {
    int x = 5, y = 10;
    printf("Before: x=%d, y=%d\n", x, y); // x=5, y=10
    swap(&x, &y);
    printf("After: x=%d, y=%d\n", x, y);  // x=10, y=5
}

```

Example 1: Swap with Pointers (Pass by Reference)

```

void minmax(int arr[], int n, int *min, int *max) {
    *min = arr[0];
    *max = arr[0];

    for (int i = 1; i < n; i++) {
        if (arr[i] < *min) *min = arr[i];
        if (arr[i] > *max) *max = arr[i];
    }
}

```

```

}

int main() {
    int arr[] = {3, 1, 4, 1, 5, 9, 2, 6};
    int min, max;
    minmax(arr, 8, &min, &max);
    printf("Min: %d, Max: %d\n", min, max); // Min: 1, Max: 9
}

```

Example 2: Returning Multiple Values via Pointers

```

#include <stdio.h>

int value = 100; // Global

int main() {
    int value = 200; // Local, shadows global

    printf("Local: %d\n", value); // 200 (local)
    printf("Global: %d\n", ::value); // 100 (global)

    {
        int value = 300; // Inner block
        printf("Inner: %d\n", value); // 300
        printf("Global: %d\n", ::value); // Still 100
    }

    printf("Back to: %d\n", value); // 200 (back to local)
}

```

Example 3: Scope Resolution

```

#include <stdio.h>

int IDgenerator() {
    static int nextID = 1000; // Retains value between calls!
    return nextID++;
}

int main() {
    printf("%d\n", IDgenerator()); // 1000
    printf("%d\n", IDgenerator()); // 1001
    printf("%d\n", IDgenerator()); // 1002
}

```

Example 4: Static Counter Pattern

```

#include <stdio.h>
#include <stdlib.h>

int *createArray(int size, int initialValue) {
    int *arr = (int*)malloc(size * sizeof(int));
    if (arr == NULL) {
        return NULL; // Allocation failed!
    }
    for (int i = 0; i < size; i++) {
        arr[i] = initialValue;
    }
    return arr; // Caller responsible for freeing!
}

int main() {
    int *nums = createArray(5, 42);
    if (nums == NULL) return 1;

    for (int i = 0; i < 5; i++) {
        printf("%d ", nums[i]); // 42 42 42 42 42
    }

    free(nums); // DON'T FORGET!
    nums = NULL; // Prevent dangling pointer
}

```

Example 5: Memory Management Complete

Execution Stack Walkthrough

Function call: main() calls foo(5), foo() calls bar(10)

Stack (top at bottom, growing down):

HIGH ADDRESSES

```

+-----+
| Return addr (main)          | <- bar returns to foo
+-----+
| bar param: x = 10           |
+-----+
| bar local: y = 0            |
+-----+
| Return addr (foo)           | <- foo returns to main
+-----+
| foo param: n = 5             |
+-----+
| foo local: z = 0             |

```

```

+-----+
| main local: a = 0 |
+-----+
LOW ADDRESSES

```

When bar returns:

```

+-----+
| Return addr (foo) |
+-----+
| foo param: n = 5 |
+-----+
| foo local: z = 0 |
+-----+
| main local: a = 0 |
+-----+

```

When foo returns:

```

+-----+
| main local: a = 0 |
+-----+

```

Practice Questions

MCQ 1: What is printed?

```

void change(int x) { x = 20; }
int main() {
    int num = 10;
    change(num);
    printf("%d", num);
}

```

- A) 10
- B) 20
- C) Garbage
- D) Error

MCQ 2: What is a memory leak? - A) Accessing freed memory - B) Allocating without freeing - C) Stack overflow - D) Uninitialized variable

MCQ 3: Where is a static local variable stored? - A) Stack - B) Heap - C) Data segment - D) Code segment

Short Answer: Explain the difference between pass by value and pass by reference. When would you use pass by reference?

Board Coding: Write a function that finds the largest and second largest elements in an array, using pass by reference.

Oral Explanation: Draw the memory layout showing stack, heap, and static regions. Explain what happens when a function is called and returns.

Solutions

MCQ 1: A) 10 – num is passed by value, original unchanged

MCQ 2: B) Allocating without freeing – memory never returned to system

MCQ 3: C) Data segment – static storage persists

Short Answer: Pass by value passes a copy; changes don't affect original. Pass by reference passes address; changes affect original. Use pass by reference when you need to modify multiple values, or when copying data would be inefficient.

Board Coding:

```
void findTwoLargest(int arr[], int n, int *max1, int *max2) {
    *max1 = arr[0];
    *max2 = arr[1];
    if (*max2 > *max1) {
        int temp = *max1;
        *max1 = *max2;
        *max2 = temp;
    }
    for (int i = 2; i < n; i++) {
        if (arr[i] > *max1) {
            *max2 = *max1;
            *max1 = arr[i];
        } else if (arr[i] > *max2) {
            *max2 = arr[i];
        }
    }
}
```

Cheat-Sheet

Concept	Key Points
Stack	Grows down, automatic cleanup, fast, limited size
Heap	Grows up, manual (malloc/free), larger, scattered
Static	Whole program lifetime, stored in data segment
Pass by Value	Copy passed, original unchanged
Pass by Reference	Address passed, original can be modified
Scope	Where identifier is visible (block, function, file)
Static local	Retains value between calls
malloc/free	Allocate/free heap memory in C

Review Checklist

- ☐ **Storage classes:** Can you identify stack, heap, static variables?
- ☐ **Pass by value:** Can you explain why original is unchanged?
- ☐ **Pass by reference:** Can you show how pointers modify original?
- ☐ **malloc/free:** Do you pair every allocation with free?
- ☐ **Dangling pointers:** Do you set to NULL after free?
- ☐ **Scope:** Can you identify variable visibility in nested blocks?
- ☐ **Static:** Do you understand why static local retains value?
- ☐ **Lifetime:** Can you trace when variables are created/destroyed? # Chapter 6: The Concept of Exceptions

6.1 Raising and Propagation of Exceptions

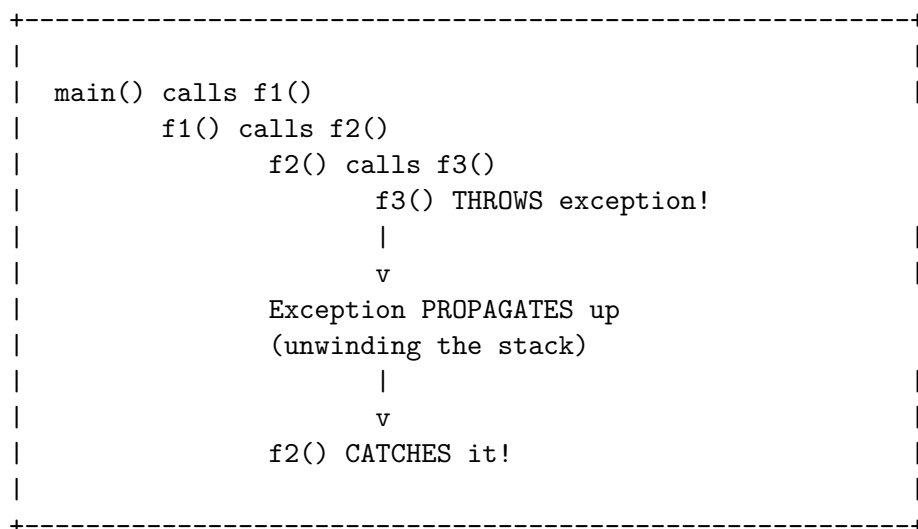
Definition / Theoretical Background

An **exception** is an event that occurs during program execution that disrupts the normal flow of instructions. When something goes wrong (division by zero, null pointer, file not found), the normal error handling would be messy with multiple return codes. Exceptions provide a cleaner way to handle this.

Key Concepts:

Term	Meaning
Raise/Throw	Signal that an error occurred
Propagation	Exception moves up the call stack until caught
Catch/Handle	Code that responds to the exception
Uncaught	Exception reaches top-level, program terminates

Exception Flow Diagram



Stack unwinding:

f3() throws -> f2() catches -> normal flow resumes in f2()

If f2() didn't catch, would go to f1(), then main(), then crash

How Exceptions Are Raised

```
// C++ way
void divide(int a, int b) {
    if (b == 0) {
        throw "Division by zero!"; // Raise exception
    }
    cout << a / b << endl;
}

// Java way
void divide(int a, int b) {
    if (b == 0) {
        throw new ArithmeticException("Division by zero"); // Raise
    }
    System.out.println(a / b);
}

// Python way
def divide(a, b):
    if b == 0:
        raise ValueError("Division by zero") # Raise exception
    return a / b
```

Exception Propagation (Stack Unwinding)

```
// C++ Example: Exception propagation
void level3() {
    cout << "Level 3: About to throw..." << endl;
    throw runtime_error("Error in level 3!");
    cout << "Level 3: After throw (never reached)" << endl;
}

void level2() {
    cout << "Level 2: Calling level3..." << endl;
    level3(); // Exception propagates THROUGH level2!
    cout << "Level 2: After level3 (never reached)" << endl;
}

void level1() {
    cout << "Level 1: Calling level2..." << endl;
    level2(); // Exception still propagating!
    cout << "Level 1: After level2 (never reached)" << endl;
}
```

```

int main() {
    try {
        cout << "main: Calling level1..." << endl;
        level1();
        cout << "main: After level1 (never reached)" << endl;
    }
    catch (const exception& e) {
        cout << "CAUGHT: " << e.what() << endl;
    }
    cout << "main: After catch block" << endl;    // This runs!
}

```

*/**

Output:

main: Calling level1...

Level 1: Calling level2...

Level 2: Calling level3...

Level 3: About to throw...

CAUGHT: Error in level 3!

main: After catch block

Notice how the exception SKIPS all the "After" print statements because it propagates through the call stack!

**/*

Python Exception Propagation

```

def level3():
    print("Level 3: About to raise...")
    raise ValueError("Error in level 3!")    # Raise
    print("Level 3: After raise (never reached)")

def level2():
    print("Level 2: Calling level3...")
    level3()    # Propagates through!
    print("Level 2: After level3 (never reached)")

def level1():
    print("Level 1: Calling level2...")
    level2()    # Propagates through!
    print("Level 1: After level2 (never reached)")

try:
    print("main: Calling level1...")
    level1()
    print("main: After level1 (never reached)")
except ValueError as e:

```

```

    print(f"CAUGHT: {e}")

print("main: After except block")  # This runs!

```

Edge Cases & Traps

Trap 1: Forgetting to Catch (Program Crashes!)

```

// C++ - No catch block = terminate() called!
void dangerous() {
    throw runtime_error("Oops!");  // No one catching this!
}
// main() doesn't have try-catch = program crashes

```

[WARNING] **Professor Trap:** If exception reaches top-level without a handler, program terminates!

Trap 2: Caught by Wrong Handler

```

try {
    riskyFunction();
}
catch (int) {  // Catching int, but string was thrown!
    // This won't catch a thrown string!
}
// Exception will propagate to outer handlers or crash!

```

[WARNING] **Professor Trap:** Catch blocks must match the exception type being thrown!

Trap 3: Catch-All Anti-Pattern

```

// BAD: Catching everything
catch (...) {
    // Catches ANY exception - even non-errors!
    // No information about what went wrong!
    cout << "Something went wrong?" << endl;
}

```

[WARNING] **Professor Trap:** Catching catch(...) hides errors - avoid unless necessary!

Trap 4: Throwing in Destructor (C++)

```

class Cleanup {
    ~Cleanup() {
        throw "Error in destructor!";  // BAD! While stack unwinding!
    }
};

void bad() {
    Cleanup c;
    throw "Original error";  // Destructor called during stack unwinding
}  // double fault - terminate() called!

```

[WARNING] **Professor Trap:** Throwing exceptions in destructors during stack unwinding causes `terminate()!`

Trap 5: Uncaught Exception in Constructor (C++)

```
class Bad {
    int *ptr;
public:
    Bad() {
        ptr = new int[100];
        throw "Allocation failed!"; // ptr leaked! No cleanup!
    }
};
// Object never fully constructed, destructor won't run!
```

[WARNING] **Professor Trap:** If constructor throws, destructor is never called for that object!

6.2 Throwing and Handling Exceptions

The try-catch-finally Pattern

```
// C++ pattern
try {
    // Code that might throw
    riskyOperation();
}
catch (ExceptionType1& e1) {
    // Handle e1
}
catch (ExceptionType2& e2) {
    // Handle e2
}
catch (...) {
    // Handle anything else (catch-all)
}
finally {
    // Cleanup (in C++, use RAII or scope guards instead!)
}
```

```
// Java pattern
try {
    // Code that might throw
    riskyOperation();
} catch (IOException e) {
    // Handle IOException
} catch (RuntimeException e) {
    // Handle RuntimeException
} finally {
```

```

    // ALWAYS executes (even if no exception)
    cleanup();
}

# Python pattern
try:
    risky_operation()
except ValueError as e:
    # Handle ValueError
    print(f"ValueError: {e}")
except (TypeError, KeyError) as e:
    # Handle multiple types
    print(f"Error: {e}")
except:
    # Catch-all (bare except)
    print("Something went wrong")
    raise # Re-raise the exception
finally:
    # ALWAYS executes
    cleanup()

```

Syntax Comparison Across Languages

Language	Throw	Catch	Finally	Notes
C++	throw expr	catch(type e)	RAII/ScopeGuard	No finally keyword
Java	throw new Ex()	catch(Ex e)	finally {}	Checked exceptions
Python	raise Ex()	except Ex as e	finally:	Bare raise re-raises
JavaScript	throw expr	catch(e)	finally{}	try-catch-finally
C#	throw new Ex()	catch(Ex e)	finally{}	When filters

Custom Exception Classes

```

// C++ custom exception
class MyException : public exception {
    string message;
public:
    MyException(const string& msg) : message(msg) {}
    const char* what() const noexcept override {
        return message.c_str();
    }
};

throw MyException("Something went wrong!");

```

```

// Java custom exception
public class MyException extends Exception {

```

```

    public MyException(String message) {
        super(message);
    }
}

throw new MyException("Something went wrong!");

# Python custom exception
class MyException(Exception):
    pass

raise MyException("Something went wrong!")

```

Best Practices for Exception Handling

```

// GOOD: Specific catch
try {
    openFile("data.txt");
}
catch (FileNotFoundException& e) {
    cout << "File not found: " << e.what() << endl;
}
catch (PermissionDeniedException& e) {
    cout << "Access denied: " << e.what() << endl;
}

```

DO: Catch Specific Exceptions

```

// BAD: Too broad
catch (...) {
    // Hides bugs! What went wrong?
    cout << "An error occurred" << endl;
}

```

DON'T: Catch Everything

The finally Block (Cleanup)

Purpose: Ensure cleanup code runs regardless of whether exception occurred.

```

// Java/C#/Python finally
FileReader reader = null;
try {
    reader = new FileReader("data.txt");
    // Read file...
}
catch (IOException e) {

```

```

        e.printStackTrace();
    }
    finally {
        if (reader != null) {
            try { reader.close(); } catch (IOException e) {}
        }
        // This ALWAYS runs, even if exception thrown
    }

# Python with statement (preferred over manual finally)
with open("data.txt") as f:
    data = f.read()
# File automatically closed, even if exception occurs!

// C++ uses RAII instead of finally
class FileGuard {
    FILE* f;
public:
    FileGuard(const char* name) { f = fopen(name, "r"); }
    ~FileGuard() { if (f) fclose(f); } // Called automatically!
};

void readFile() {
    FileGuard guard("data.txt"); // RAII - cleanup automatic
    // If exception thrown, destructor called during unwinding
}

```

6.3 Various Kinds of Exceptions

Exception Hierarchy in C++

```

exception (base)
|
+-- logic_error
|   +-- invalid_argument
|   +-- length_error
|   +-- out_of_range
|
+-- runtime_error
    +-- range_error
    +-- overflow_error
    +-- underflow_error
    +-- filesystem_error (C++17)

```

Common Exception Types

Exception Type	When Thrown	Example
<code>invalid_argument</code>	Invalid parameter	<code>stoi("abc")</code>
<code>out_of_range</code>	Index out of bounds	<code>vector.at(100)</code> on 10-element vector
<code>length_error</code>	Exceeds max size	<code>vector.resize(max+1)</code>
<code>bad_alloc</code>	Memory allocation failed	<code>new</code> fails
<code>null_pointer</code>	Null pointer dereference	Calling on <code>nullptr</code>
<code>overflow_error</code>	Arithmetic overflow	<code>INT_MAX + 1</code>
<code>domain_error</code>	Math domain error	<code>sqrt(-1)</code>

Java Exception Hierarchy

Throwable

```

|
+-- Error (unchecked, serious problems)
|   +-- OutOfMemoryError
|   +-- StackOverflowError
|
+-- Exception
    +-- RuntimeException (unchecked)
        |   +-- NullPointerException
        |   +-- IndexOutOfBoundsException
        |   +-- ArithmeticException
        |
    +-- IOException (checked)
        +-- FileNotFoundException
        +-- EOFException

```

Checked vs Unchecked Exceptions (Java)

```

// CHECKED exceptions - must be caught or declared
public void readFile() throws IOException {
    FileReader f = new FileReader("data.txt");
    // IOException MUST be handled
}

// UNCHECKED exceptions - can occur anywhere
public int divide(int a, int b) {
    if (b == 0) {
        throw new ArithmeticException("Division by zero");
        // ArithmeticException is unchecked, no throws required
    }
    return a / b;
}

```

Python Built-in Exceptions

```
BaseException
  +-- SystemExit
  +-- KeyboardInterrupt
  +-- GeneratorExit
  +-- Exception
      +-- StopIteration
      +-- ArithmeticError
      |   +-- FloatingPointError
      |   +-- OverflowError
      |   +-- ZeroDivisionError
      +-- LookupError
      |   +-- IndexError
      |   +-- KeyError
      +-- ValueError
      +-- TypeError
      +-- NameError
      +-- RuntimeError
```

Common Error Scenarios and Their Exceptions

```
// C++ - floating point
double a = 5.0, b = 0.0;
double result = a / b; // Result: inf (no exception!)

// C++ - integer - UNDEFINED BEHAVIOR!
int x = 5, y = 0;
int result = x / y; // UNDEFINED! Don't do this!

// Java - throws ArithmeticException
int x = 5, y = 0;
int result = x / y; // Throws ArithmeticException
```

```
# Python - raises ZeroDivisionError
try:
    x = 5 / 0
except ZeroDivisionError:
    print("Cannot divide by zero!")

# But float division gives inf
x = 5.0 / 0.0 # x = inf, no exception!
```

Division by Zero

```
// C++ - undefined behavior, usually crashes  
int *ptr = nullptr;  
*ptr = 42; // CRASH!
```

```
// Java - throws NullPointerException  
String s = null;  
s.length(); // Throws NullPointerException
```

```
# Python - raises AttributeError  
s = None  
s.upper() # AttributeError: 'NoneType' object has no attribute 'upper'
```

Null Pointer / Reference

```
// C++ - undefined behavior!  
vector<int> v = {1, 2, 3};  
int x = v.at(10); // throws out_of_range  
int y = v[10]; // UNDEFINED! No bounds check!
```

```
// Java - throws IndexOutOfBoundsException  
ArrayList<Integer> list = new ArrayList<>();  
list.add(1);  
list.get(10); // Throws IndexOutOfBoundsException
```

```
# Python - raises IndexError  
lst = [1, 2, 3]  
lst[10] # IndexError: list index out of range  
  
# But slicing is safe!  
lst[0:100] # Returns [1, 2, 3] - doesn't error!
```

Index Out of Bounds

```
// C++ - invalid_argument  
string s = "abc";  
int x = stoi(s); // throws invalid_argument: stoi
```

```
// Java - NumberFormatException  
int x = Integer.parseInt("abc"); // throws NumberFormatException
```

```
# Python - ValueError
int("abc") # ValueError: invalid literal for int()
```

Invalid Type Conversions

Error Handling in Different Language Contexts

```
// C uses return codes and errno instead of exceptions
#include <errno.h>
#include <string.h>

FILE *f = fopen("nonexistent.txt", "r");
if (f == NULL) {
    printf("Error opening file: %s\n", strerror(errno));
    // errno set by fopen: ENOENT (no such file), EACCES (permission), etc.
}

// Or check return value
int result = divide(10, 0);
if (result == -1) {
    // Handle error
}

// But programmers often forget to check!
divide(10, 0); // Undefined behavior if we don't check!
```

Error Handling in C (No Exceptions!)

```
// Go uses error as return value
func divide(a, b int) (int, error) {
    if b == 0 {
        return 0, fmt.Errorf("division by zero")
    }
    return a / b, nil
}

result, err := divide(10, 0)
if err != nil {
    fmt.Println("Error:", err)
}
```

Error Handling in Go (Multiple Return Values)

```
// Rust uses Result enum
enum Result<T, E> {
```

```

    Ok(T),
    Err(E),
}

fn divide(a: i32, b: i32) -> Result<i32, String> {
    if b == 0 {
        Err("division by zero".to_string())
    } else {
        Ok(a / b)
    }
}

match divide(10, 0) {
    Ok(n) => println!("Result: {}", n),
    Err(e) => println!("Error: {}", e),
}

```

Error Handling in Rust (Result Type)

Exception Safety Levels (C++)

Level	Guarantee	What It Means
No-throw	Never throws	Operations always succeed
Strong	Succeeds or rolls back	Either complete or unchanged
Basic	Objects still usable	No leaks, but may be in valid but modified state
No guarantee	May be broken	Don't use!

```

// Example: No-throw guarantee
class Widget {
public:
    int& operator[](size_t i) noexcept {
        return data[i]; // Never throws!
    }
};

```

Edge Cases & Traps

Trap 1: Swallowed Exceptions

```

// BAD - exception disappears!
catch (Exception e) {
    // Doing nothing with e!
    // Bug is hidden!
}

// This is worse:

```

```
catch (...) {
    // Ignoring ALL exceptions - very bad!
}
```

Trap 2: Rethrowing Incorrectly

```
// WRONG - copies exception (slicing!)
catch (exception& e) {
    throw e; // Throws COPY, may lose info!
}

// CORRECT - rethrow original
catch (exception& e) {
    throw; // Rethrows ORIGINAL exception
}
```

Trap 3: Exception in Constructor - Resource Leak

```
class BadResource {
    FILE* file;
public:
    BadResource(const char* name) {
        file = fopen(name, "r");
        if (somethingBad) {
            throw "Error!"; // file leaked! destructor won't run!
        }
    }
};

// FIXED - use RAII
class GoodResource {
    FileGuard file; // RAII - destructor called even if exception!
public:
    GoodResource(const char* name) : file(name) {
        if (somethingBad) {
            throw "Error!"; // FileGuard destructor called, file closed!
        }
    }
};
```

Trap 4: Exception During Exception Handling

```
try {
    risky();
}
catch (...) {
    cleanup(); // If cleanup() throws, terminate() is called!
    // Best to make cleanup() noexcept
}
```

Trap 5: Destructors Throwing During Stack Unwinding (C++)

```
class Dangerous {
public:
    ~Dangerous() noexcept(false) {
        throw "Error in destructor!"; // BAD during unwinding!
    }
};

void foo() {
    Dangerous d;
    throw "From foo"; // Destructor called, throws, terminate()!
}
```

[WARNING] **Professor Trap:** Never throw from destructor during stack unwinding!

Examples

```
#include <iostream>
#include <stdexcept>
using namespace std;

void test(int choice) {
    try {
        switch (choice) {
            case 1: throw invalid_argument("Bad argument");
            case 2: throw out_of_range("Out of range");
            case 3: throw overflow_error("Overflow!");
            case 4: throw bad_alloc();
            default: throw runtime_error("Unknown error");
        }
    }
    catch (invalid_argument& e) {
        cout << "invalid_argument: " << e.what() << endl;
    }
    catch (out_of_range& e) {
        cout << "out_of_range: " << e.what() << endl;
    }
    catch (overflow_error& e) {
        cout << "overflow_error: " << e.what() << endl;
    }
    // catch (bad_alloc& e) { ... } // Would catch memory errors
    catch (exception& e) { // Catches ALL std::exception derived types
        cout << "Other exception: " << e.what() << endl;
    }
}
```

Example 1: Exception Hierarchy Understanding

```

#include <iostream>
#include <memory>
using namespace std;

class FileHandler {
    FILE* f;
public:
    FileHandler(const char* name) {
        f = fopen(name, "r");
        if (!f) throw runtime_error("Cannot open file");
    }
    ~FileHandler() {
        if (f) fclose(f); // ALWAYS called, even with exceptions!
        cout << "File closed" << endl;
    }
};

void processFile() {
    FileHandler file("data.txt"); // Opens file
    // If exception thrown here...
    // ~FileHandler() called automatically, file closed!
}

```

Example 2: RAII Resource Management

```

# Python 3 - exception chaining
try:
    int("not a number")
except ValueError as e:
    raise TypeError("Failed to process") from e

# Output shows both exceptions!
# TypeError: Failed to process
# The above exception was the direct cause of the following exception:
# ValueError: invalid literal for int() with base 10: 'not a number'

```

Example 3: Python Exception Chaining

```

try {
    // Risky operations
    String s = args[0];
    int x = Integer.parseInt(s);
    int result = 10 / x;
    int[] arr = new int[5];
    arr[x] = 100;
}

```

```

}
catch (ArrayIndexOutOfBoundsException e) {
    System.out.println("Index error: " + e.getMessage());
}
catch (NumberFormatException e) {
    System.out.println("Not a number: " + e.getMessage());
}
catch (ArithmeticException e) {
    System.out.println("Math error: " + e.getMessage());
}
catch (Exception e) {
    System.out.println("Other error: " + e.getMessage());
}
finally {
    System.out.println("Always runs!");
}

```

Example 4: Java Multiple Catch

Practice Questions

MCQ 1: What happens if an exception is thrown and not caught? - A) Program continues normally - B) Program crashes - C) Exception is ignored - D) Returns to previous function

MCQ 2: In C++, if a constructor throws, what happens to the destructor? - A) Called normally - B) Never called for that object - C) Called with error - D) Undefined

MCQ 3: Which is the correct way to rethrow an exception in C++? - A) `throw e;` - B) `throw;` - C) `throw new Exception(e);` - D) `rethrow(e);`

Short Answer: Explain the difference between checked and unchecked exceptions in Java.

Board Coding: Write a C++ function that opens a file and uses RAII to ensure it's closed even if an exception is thrown.

Oral Explanation: Draw a diagram showing exception propagation through nested function calls. What happens if no function catches the exception?

Solutions

MCQ 1: B) Program crashes (or calls `std::terminate`)

MCQ 2: B) Never called for that object - object was never fully constructed

MCQ 3: B) `throw;` - rethrows original exception; `throw e` creates a copy which may slice

Short Answer: Checked exceptions must be declared in the method signature (`throws`) and must be caught or declared. Unchecked exceptions (`RuntimeException` and subclasses) don't need to be declared or caught. Checked exceptions represent recoverable conditions (`IOException`); unchecked represent programming errors (`NullPointerException`).

Cheat-Sheet

Concept	Key Points
Throw/Raise	Signal an error condition
Propagation	Exception moves up call stack until caught
Catch	Handle specific exception type
Finally	Cleanup code that always runs
Stack unwinding	Destroying stack frames while propagating
RAII	C++ uses constructors/destructors for cleanup
Checked exceptions	Must be caught (Java)
Unchecked exceptions	Don't need to be caught (RuntimeException)

Review Checklist

- ☐ **Propagation:** Can you trace exception flow through nested calls?
- ☐ **Catch specificity:** Do you catch specific exceptions first?
- ☐ **RAII:** Do you use RAII to avoid needing finally?
- ☐ **Destructor safety:** Can you avoid throwing from destructors?
- ☐ **Re-throw:** Do you use `throw;` not `throw e;`?
- ☐ **Resource cleanup:** Do you use try-finally or RAII?
- ☐ **Exception types:** Do you know common exception types per language?